COLLEGE DE PARIS SUPERIEUR

Information Technology

Tronc commun

RÉPUBLIQUE TOGOLAISE

Travail – Liberté – Patrie

INTRODUCTION À LA PROGRAMMATION

en langage Python

De l'algorithmique à l'écriture de scripts Python

Variables, conditions, boucles, fonctions,
structures de données, fichiers, exceptions

Python IDE Débogage

Niveau : Bachelor 1 (B1) – Licence en IT

Dr. AFENYIVEH Dzo M. Serge

dzo.serge@gmail.com / d.afenyiveh@univkara.com

(+228) 91 75 98 33 / 98 35 23 31

18 décembre 2025

Année académique : 2025–2026

Sommaire

SYLLABUS DU COURS **1**

1 Module 1 : Introduction et environnement Python **6**
1.1 Historique, philosophie et caractéristiques de Python 6
1.2 Installation et gestion des packages Python 9
1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python 14
1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet 17
1.5 Structure d'un programme Python : Organiser votre premier code. 22
1.6 Variables et types de données de base : Les fondations de l'information IT 25
1.7 Les Opérateurs en Python : Manipuler les données..... 32
1.8 Travaux Pratiques (TP) du Module 1 40
1.9 Quiz du Module 1 : Testez vos connaissances..... 45

2 Structures de Contrôle : Logique et Automatisation **48**
2.1 Expressions Booléennes : La base de la logique..... 49
2.2 Structures Conditionnelles : Prendre des décisions 51
2.3 La Boucle `for` : Itérer sur des séquences 56
2.4 La Boucle `while` : Répéter tant que... 58
2.5 Contrôle du Flux : `break`, `continue`, `pass` 61
2.6 Cas Pratiques IT 62
2.7 Travaux Pratiques du Module 2 64
2.8 Quiz du Module 2..... 66

3 Fonctions en Python : Modularité et Réutilisabilité **68**
3.1 Définition et Appel de Fonctions..... 70
3.2 Retour de Valeurs avec `return` 72
3.3 Types de Paramètres 74
3.4 Portée des Variables (Scope) 77
3.5 Fonctions Intégrées (Built-in) 78
3.6 Modules Standards 79

3.7 Fonctions Lambda (Anonymes) 81
3.8 Documentation des Fonctions (Docstrings) 82
3.9 Travaux Pratiques du Module 3 84
3.10 Quiz du Module 3 87

Syllabus du Cours

Programmation Python

Introduction à la Programmation en Langage Python

Bachelor 1 (B1) – Tronc commun

Description du Cours

Ce cours initie les étudiants aux concepts fondamentaux de la programmation en utilisant **Python**, un langage polyvalent et accessible, largement utilisé en *science des données*, *développement web* et *automatisation*.

Python **Scripts** **Débogage** **Mini-projets**

Informations Générales

Intitulé	Introduction à la Programmation en Langage Python
# Code	B1COM0304
Niveau	Bachelor 1 (B1) – Tronc commun
Durée	24 heures (CM + TD + TP)
Crédits	2
Semestre	Semestre 1
Prérequis	› Introduction à l'Algorithmique et structure de données

Objectifs Pédagogiques

À l'issue de ce cours, l'étudiant doit être capable de :

- ✅ Comprendre la syntaxe et les concepts fondamentaux du langage Python.
- ✅ Écrire et exécuter des scripts Python simples pour résoudre des problèmes.
- ✅ Utiliser les structures de données principales : listes, dictionnaires, tuples et ensembles.
- ✅ Appliquer les structures de contrôle pour la logique de programmation.
- ✅ Déboguer et tester des programmes Python.

Contenu Pédagogique

1. Introduction au langage Python

- Historique et applications du langage Python
- Installation de Python et prise en main de l'environnement de développement (IDE)
- Structure d'un programme Python
- Types de données de base et variables

2. Structures de contrôle

- Les conditions : `if`, `else`, `elif`
- Les boucles : `for`, `while`
- Utilisation des boucles pour itérer sur des structures de données

3. Fonctions en Python

- Définition et appel de fonctions
- Paramètres et retour de valeurs
- Portée des variables (locale et globale)
- Fonctions intégrées et modules standards (`math`, `random`, etc.)

4. Structures de données

- Listes et opérations associées (ajout, suppression, tri, etc.)
- Dictionnaires : création, accès et manipulation
- Tuples et ensembles : caractéristiques et utilisation
- Applications pratiques des structures de données

5. Introduction à la manipulation des fichiers

- Lecture et écriture dans les fichiers
- Gestion des erreurs lors de l'accès aux fichiers
- Utilisation de fichiers pour stocker et récupérer des données

6. Introduction au débogage et bonnes pratiques

- Techniques de débogage de base
- Gestion des exceptions (`try`, `except`)
- Introduction à l'utilisation des bibliothèques
- Documentation et structuration du code
- Bonnes pratiques pour écrire un code Python clair et lisible

Approches Pédagogiques

- Exercices de programmation à chaque séance pour pratiquer les concepts.
- Utilisation de notebooks interactifs (Jupyter/Colab) pour faciliter l'expérimentation.
- Travaux dirigés orientés vers la résolution de problèmes pratiques avec Python.
- Mini-projets pour renforcer les compétences en fin de cours.

Méthode d'évaluation

- **Contrôle(s) continu(s) (TD ou TP)** : **60%** de la note finale
- **Examen final ou Projet Final** : **40%** de la note finale

Références Bibliographiques

[1] **Zelle, J.** *Python Programming : An Introduction to Computer Science.* Franklin, Beedle & Associates, 2010.

[2] **Matthes, E.** *Python Crash Course.* No Starch Press, 2016.

[3] **Sweigart, A.** *Automate the Boring Stuff with Python.* No Starch Press, 2015.

[4] **Lutz, M.** *Learning Python.* O'Reilly Media, 2013.

> *« Programmer, c'est apprendre à décomposer un problème en étapes simples et vérifiables. »*

CHAPITRE I

MODULE 1 : INTRODUCTION ET ENVIRONNEMENT PYTHON

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- ✓Connaître l'historique, la philosophie et les principes fondamentaux de Python.
- ✓Comprendre la place centrale de Python dans les métiers de l'IT : développement web, data science, automatisation, etc.
- ✓Maîtriser l'installation de Python et la gestion des bibliothèques (`pip`, `conda`).
- ✓Utiliser la console interactive (REPL) pour des tests rapides et exécuter des scripts Python.
- ✓Interagir avec l'utilisateur via les fonctions `print()` et `input()`.
- ✓Choisir et configurer un environnement de développement (IDE) adapté à vos projets IT.

1.1 Historique, philosophie et caractéristiques de Python

Python est un langage de programmation **de haut niveau**, **interprété**, **orienté objet** et **multiparadigme**. Son objectif central est la **lisibilité** : un code plus facile à relire se maintient mieux et coûte moins cher à faire évoluer, ce qui est déterminant en **Information Technology**. Le langage a été initié par **Guido van Rossum** en 1989 au *CWI* (Centrum Wiskunde & Informatica, Pays-Bas) et rendu public en 1991.

La philosophie *"batteries included"* signifie que Python fournit dès l'installation une bibliothèque standard riche (fichiers, JSON, réseau, dates, tests,

etc.). En IT, cela se traduit par un gain immédiat : moins de dépendances à ajouter pour produire un script utile.

Le Zen de Python : principes utiles en ingénierie logicielle

Le *Zen de Python* (Tim Peters) est accessible via `import this`. Il sert de guide pour écrire du code exploitable en équipe :

- **Beau est mieux que laid. Utilité IT :** La lisibilité réduit le temps de revue, de maintenance et le taux d'erreurs.
- **Explicite est mieux qu'implicite. Utilité IT :** Un comportement explicite diminue les bugs et facilite le débogage en production.
- **Simple est mieux que complexe. Utilité IT :** La complexité augmente les risques (pannes, vulnérabilités, coûts).
- **La lisibilité compte. Utilité IT :** L'onboarding et la collaboration (Agile/DevOps) deviennent plus rapides.

1.1.1 Sous le capot : interprété vs compilé (utile pour Dev/-Debug)

En informatique, la manière dont un langage est exécuté est fondamentale pour la performance, le déploiement et le débogage. **Python est un langage interprété**. Contrairement aux langages compilés (comme C++, Java où le code source est transformé en un fichier binaire exécutable avant d'être lancé), Python exécute le code via un programme intermédiaire : l'interpréteur. Il lit, analyse et exécute chaque instruction ligne par ligne, ou en convertit le code source en un "bytecode" intermédiaire exécuté par une machine virtuelle Python.

Cette distinction a des implications directes sur votre travail en IT :

- **Cycle de développement (DevOps) :** Rapidité d'essai et de modification du code. Pas de longue phase de compilation.
- **Déploiement et Distribution :** Moins de fichiers binaires à gérer, mais nécessite un interpréteur Python sur la machine cible.
- **Débogage (Debugging) :** Facilite l'identification des erreurs, car elles sont souvent détectées à l'exécution de la ligne concernée.
- **Performance :** Généralement plus lent à l'exécution qu'un langage compilé, mais la productivité du développeur est accrue. Pour pallier cela, les bibliothèques critiques de Python sont souvent écrites en C/C++ et "liées"

(bindées) à Python.

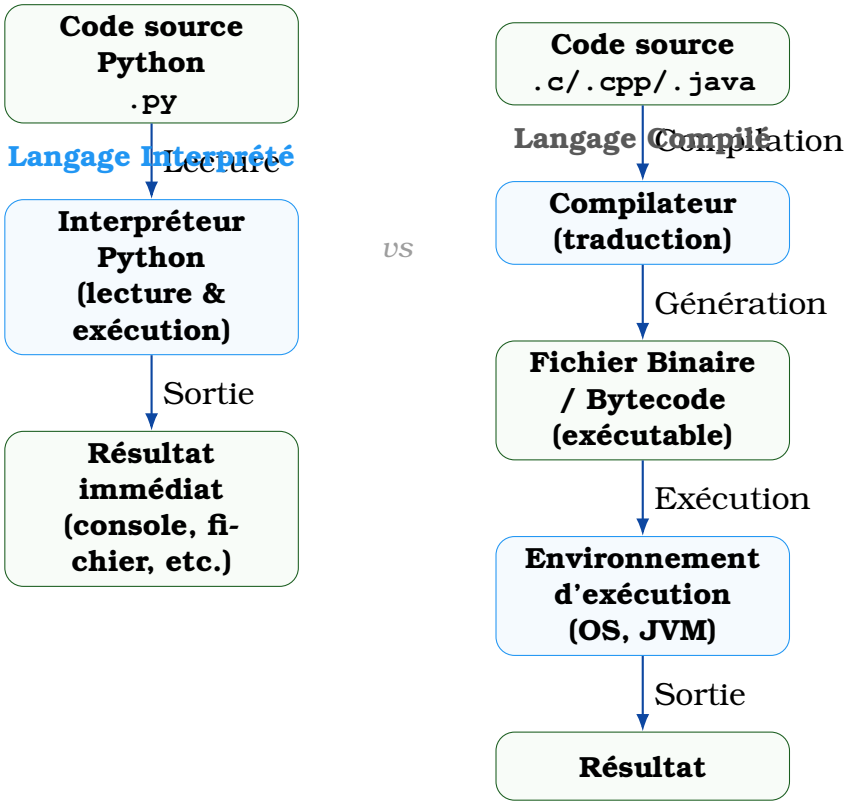


FIGURE 1.1 – *Comparaison des chaînes d'exécution : interprété (Python) vs compilé (ex : C++, Java).*

Utilité IT immédiate

Python est le choix privilégié pour les scripts d'administration système, les outils DevOps, les prototypes rapides d'applications web (APIs) ou les tests unitaires. Pour les tâches nécessitant des performances extrêmes (calculs intensifs, rendu graphique), on s'appuie sur des bibliothèques Python optimisées en C/C++ (comme NumPy) ou des architectures distribuées.

1.1.2 Carte conceptuelle : caractéristiques et bénéfices IT

Python offre un ensemble de caractéristiques qui se traduisent par des avantages directs et concrets pour le développeur et l'ingénieur IT. Comprendre cette synergie est essentiel pour exploiter pleinement le langage.

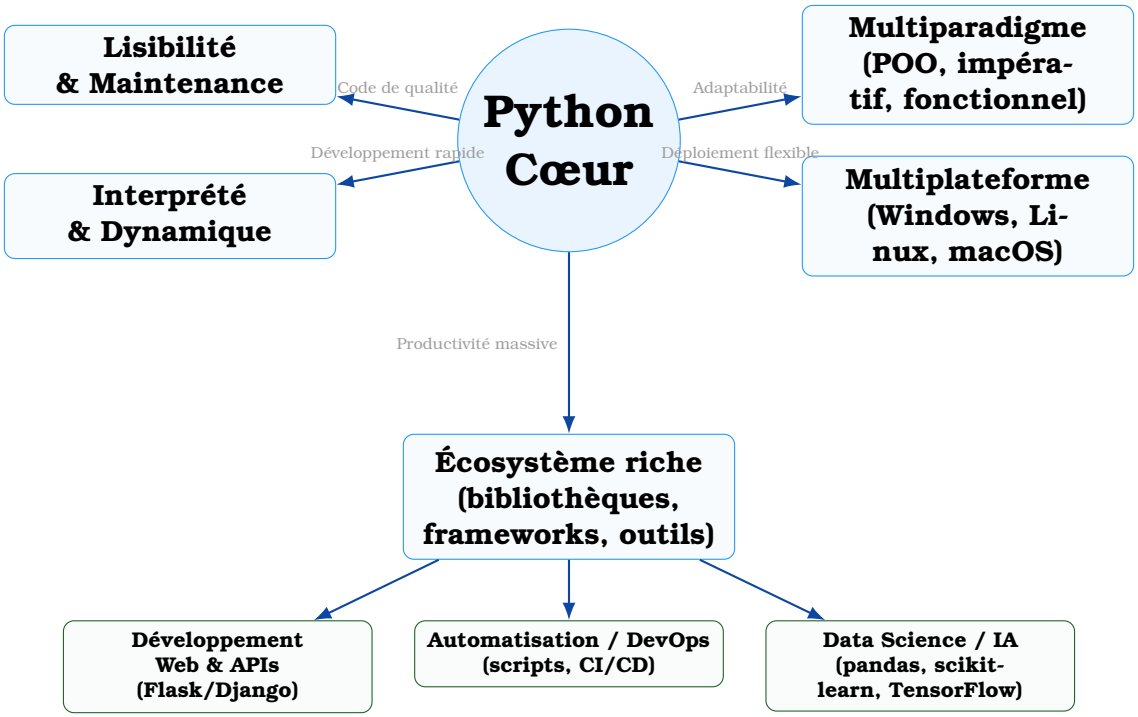


FIGURE 1.2 – *Carte conceptuelle des caractéristiques de Python et leurs bénéfices directs en IT.*

1.1.3 Chronologie : versions, maturité et rôle en IT

L'évolution de Python reflète sa capacité à s'adapter aux besoins croissants de l'industrie informatique. Comprendre cette chronologie permet d'apprécier sa robustesse actuelle.

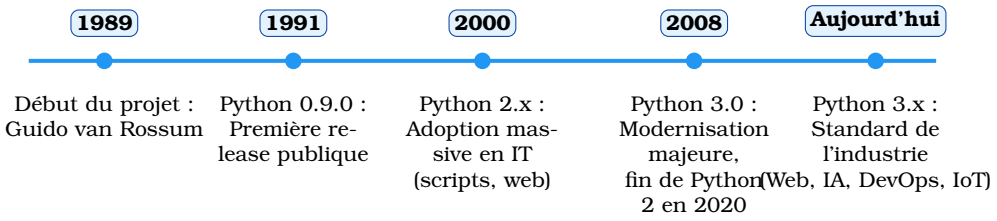


FIGURE 1.3 – *Chronologie simplifiée : Python, de son origine à son rôle de standard IT.*

1.2 Installation et gestion des packages Python

L'installation de Python et la gestion efficace de ses bibliothèques (aussi appelées "packages" ou "modules") sont des étapes fondamentales. Une configuration propre et bien organisée est la première pierre angulaire de tout projet IT réussi. Elle garantit la stabilité, la reproductibilité de vos développements et

évite les "conflits de dépendances" qui peuvent rapidement devenir un casse-tête.

1.2.1 Téléchargement et installation de Python

Choix recommandé

Pour un étudiant en **Bachelor IT**, deux voies sont réalistes :

- **Anaconda/Miniconda** : recommandé si vous visez **Data/ML** (dépendances compilées, environnements conda).
- **Python officiel** : idéal pour **scripts, web, automatisation** (léger, standard).

Option A — Anaconda / Miniconda

Profil : Data Science, ML, notebooks
Inclus : `conda`, Jupyter, libs scientifiques

Points forts :
- gestion d'environnements robuste
- dépendances natives mieux gérées
- démarrage rapide pour data/IA

Téléchargement : https://www.anaconda.com/download

Option B — Python officiel

Profil : IT général (scripts, web, automation)
Inclus : `pip` (PyPI), `venv`

Points forts :
- installation légère et standard
- excellent pour projets web/API
- workflow pro via `venv` + `pip`

Téléchargement : https://www.python.org/downloads/

FIGURE 1.4 – *Deux chemins d'installation : choisissez selon le type de projets IT visés.*

TABLE 1.1 – *Comparatif rapide : Anaconda/Miniconda vs Python officiel*

Critère	Anaconda/Miniconda	Python officiel
Gestion d'environnements	`conda` (intégré)	`venv` (standard)
Installation	plus lourde (Anaconda), légère (Miniconda)	légère
Dépendances compilées (Data/ML)	souvent plus simple	parfois plus délicat
Usage IT typique	data/IA, notebooks, calcul	scripts, web, automation, CLI
Reproductibilité	`environment.yml`	`requirements.txt`

Étapes d'installation (quel que soit le choix)

1. Télécharger l'installeur (Anaconda/Miniconda ou Python officiel).
2. Lancer l'installation et suivre l'assistant.
3. **Étape critique (Windows) :** ajouter Python au **PATH** (ou enregistrer Python par défaut).
4. Vérifier dans un terminal : `python -version`.

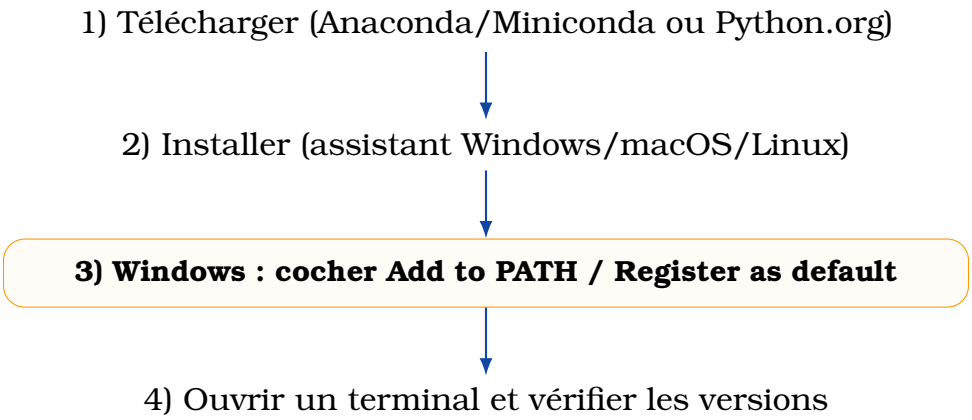


FIGURE 1.5 – *Workflow recommandé : installation + étape critique PATH + vérification.*

Vérification : confirmer que l'environnement est prêt

Après l'installation, vérifiez que les commandes sont accessibles depuis un terminal.

Terminal

```
python --version
python -c "import sys; print(sys.executable)"
pip --version
# Si Anaconda/Miniconda :
conda --version
```

Interprétation rapide

- Si `python` ou `pip` est introuvable : problème de **PATH** ou d'environnement non activé.
- `sys.executable` indique le **Python réellement utilisé** (diagnostic n°1 en cas de confusion IDE/terminal).

1.2.2 Environnements virtuels et packages : robustesse et reproductibilité

Environnement virtuel

Un **environnement virtuel** est un espace isolé contenant un Python et des bibliothèques propres à un projet. L'activation redirige `python` et `pip` vers cet espace, évitant de « polluer » l'installation globale.

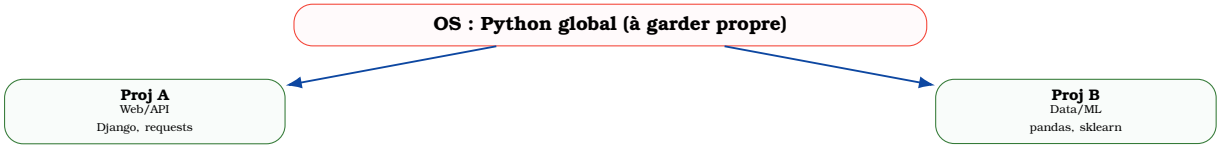


FIGURE 1.6 – *Un projet = un environnement.*

Bénéfices IT

- **Isolation** : pas de conflits de versions.
- **Reproductibilité** : recréation sur une autre machine.
- **Déploiement** : base propre CI/CD.

`pip` vs `conda` : choisir comme un ingénieur IT

TABLE 1.2 – *`pip` et `conda` : rôle et limites*

Outil	Points forts	Limites / vigilance
`pip`	Standard Python, énorme catalogue (PyPI)	gère surtout le Python ; dépendances natives parfois difficiles
`conda`	gère aussi binaires/dépendances natives ; environnements intégrés	plus « écosystème » conda ; mélange pip/conda à maîtriser

//

Terminal

```
python -m pip install requests
python -m pip install --upgrade requests
python -m pip uninstall requests
python -m pip list
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

//

Terminal

```
conda create -n dev_web python=3.11
conda activate dev_web
conda install flask numpy
conda env export > environment.yml
conda env create -f environment.yml
```

Débogage : `ModuleNotFoundError`

Cette erreur signifie : « le module n'est pas installé dans **l'environnement** qui exécute votre script ».

Méthode en 3 étapes :

1. Identifier le Python courant : `python -c "import sys; print(sys.executable)"`.
2. Installer dans ce Python : `python -m pip install nom_du_module`.
3. Relancer (et redémarrer l'IDE si nécessaire).

1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python

Le REPL (Read-Eval-Print Loop), ou interpréteur interactif Python, est votre premier outil pour dialoguer directement avec Python. Imaginez-le comme une calculatrice super-puissante où vous pouvez taper des instructions Python et obtenir un résultat immédiat. C'est l'endroit parfait pour expérimenter et tester vos premières lignes de code.

1.3.1 La console interactive Python (REPL) : Testez vos idées instantanément

Le REPL fonctionne sur un cycle simple et rapide : il **lit** ce que vous tapez, l'**évalue** (l'exécute), **affiche** le résultat, puis **boucle** pour la prochaine instruction.

Pour lancer le REPL, ouvrez un terminal (ou l'invite de commande sur Windows) et tapez `python` (ou `python3` si `python` est une version plus ancienne). Vous verrez apparaître le prompt `>>>`.

Terminal

```
python
>>> print("Bonjour, Python !") # Un message simple
Bonjour, Python !

>>> 5 + 3 # Un calcul arithmétique de base
8

>>> # Un calcul informatique simple : combien de
    kilooctets dans un mégaoctet ?
>>> 1024 * 1024
1048576

>>> "Python est " + "facile !" # On peut "coller" des
    textes
'Python est facile !'

>>> exit() # Pour quitter le REPL (ou Ctrl+D sur
    Linux/macOS, Ctrl+Z puis Entrée sur Windows)
```

Pourquoi le REPL est utile pour commencer ?

Le REPL est parfait pour :

- **Apprendre vite :** Tester des commandes Python sans avoir à créer un fichier.
- **Expérimenter :** Voir immédiatement le résultat de vos instructions.
- **Découvrir :** Explorer de nouvelles fonctions ou opérateurs.

1.3.2 Fonctions d'interaction de base : `print()` et `input()`

Pour qu'un programme soit utile, il doit pouvoir communiquer. Les fonctions `print()` et `input()` sont vos premières commandes pour que votre programme "parle" à l'utilisateur et "écoute" ce qu'il a à dire.

`print()` : Votre programme s'exprime

La fonction `print()` est la façon dont votre programme affiche du texte, des nombres, ou des résultats sur la console. C'est essentiel pour donner des informations à l'utilisateur ou pour vérifier ce que fait votre code.

In [0]:

```
## 1. Afficher un message de bienvenue
print("Bienvenue au cours de Python !")

# 2. Afficher le résultat d'un calcul
nombre_pommes = 10
nombre_oranges = 5
print("J'ai", nombre_pommes + nombre_oranges, "fruits.")

# 3. Utiliser les f-strings pour un message clair (méthode moderne)
# Les f-strings (f"...") sont pratiques pour insérer des variables directement dans le texte.
nom_utilisateur = "Alice"
print(f"Bonjour, {nom_utilisateur} ! Ravi de te voir.")
```

Out[0]:

```
Bienvenue au cours de Python ! J'ai 15 fruits. Bonjour, Alice ! Ravi de te voir.
```

L'astuce des f-strings

Les **f-strings** (chaînes de caractères formatées, préfixées par `f` comme `f"Mon texte ma_variable"`) sont la méthode la plus simple et moderne pour afficher des messages combinant du texte et des variables. Adoptez-les dès maintenant !

`input()` : Votre programme écoute l'utilisateur

La fonction `input()` permet à votre programme de demander une information à l'utilisateur et d'attendre sa réponse. La réponse est ensuite stockée dans une variable.

In [0]:

```
## 1. Demander le nom de l'utilisateur
    prenom = input("Quel est votre prénom ? ")
    print(f"Bonjour, {prenom} !")

    # 2. Demander une information numérique (attention au type !)
    annee_naissance_str = input("En quelle année êtes-vous né(e) ? ")
    # La fonction input() renvoie TOUJOURS du texte (une "chaîne de caractères").
    # Pour faire des calculs, il faut convertir ce texte en nombre.
    annee_naissance_int = int(annee_naissance_str) # Convertit le texte en nombre entier
    age_approx = 2024 - annee_naissance_int # Exemple de calcul simple
    print(f"Vous avez environ {age_approx} ans.")
```

Out[0]:

```
Quel est votre prénom ? Léa Bonjour, Léa ! En quelle année êtes-vous né(e) ? 2003 Vous avez environ 21 ans.
```

Attention au type de données avec `input()`

`input()` renvoie toujours une **chaîne de caractères** (du texte). Si vous attendez un nombre pour faire des calculs, vous devez le **convertir explicitement** en utilisant `int()` (pour les nombres entiers) ou `float()`

(pour les nombres à virgule). Si l'utilisateur tape du texte non numérique pour une conversion, votre programme produira une erreur (`ValueError`). Nous verrons comment gérer ces erreurs plus tard.

1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet

Bien que le REPL soit excellent pour l'expérimentation rapide, l'écriture et la gestion de projets Python plus complexes (applications web, scripts d'automatisation avancés, systèmes d'analyse de données) nécessitent un outil plus sophistiqué : un **IDE** (Integrated Development Environment - Environnement de Développement Intégré).

Un IDE regroupe en une seule interface des outils essentiels pour toutes les phases du développement logiciel, augmentant considérablement votre productivité et la qualité de votre code.

1.4.1 Fonctionnalités clés d'un IDE et leur utilité en IT

Un bon IDE Python offre un ensemble de fonctionnalités qui transforment votre expérience de codage :

- **Éditeur de code intelligent :**
  - **Coloration syntaxique :** Met en évidence les mots-clés, chaînes, commentaires. Facilite la lecture du code.
  - **Autocomplétion (IntelliSense) :** Suggère les noms de variables, fonctions, classes et méthodes. Réduit les erreurs de frappe et accélère l'écriture du code.
  - **Vérification d'erreurs en temps réel (Linting) :** Souligne les erreurs de syntaxe ou les avertissements de style pendant que vous écrivez. Permet de corriger les problèmes avant même d'exécuter le code.
  - **Refactoring :** Outils pour renommer des variables/fonctions à travers tout le projet, extraire du code en fonctions, etc. Maintient la qualité du code et la structure du projet.

  Utilité IT : Accélère le développement, réduit les bugs, et améliore la maintenabilité du code, des points cruciaux pour la livraison rapide et la stabilité des applications.

- **Débogueur intégré : Utilité IT :** Permet d'exécuter le code pas à pas, de définir des points d'arrêt (*breakpoints*) pour inspecter l'état des variables à un moment précis, et de détecter les erreurs logiques. Indispensable pour trouver et corriger des bugs complexes dans des systèmes IT.
- **Gestion de projet et de fichiers : Utilité IT :** Organise vos fichiers, dossiers, dépendances et environnements virtuels. Simplifie la navigation dans de grands projets et l'intégration avec les systèmes de contrôle de version (Git).
- **Console interactive intégrée : Utilité IT :** Combinaison des avantages du REPL avec la gestion du projet. Permet de tester des extraits de code dans le contexte de votre application.
- **Intégration de Terminal : Utilité IT :** Accès direct à la ligne de commande pour exécuter des scripts Python, gérer les paquets avec `pip`/`conda` ou utiliser Git, sans quitter l'IDE.

1.4.2 Les IDEs Python populaires pour les étudiants en IT

Plusieurs IDEs sont populaires et adaptés au développement Python. Votre choix dépendra de vos préférences, du type de projet et des ressources de votre machine.

PyCharm : L'IDE de référence pour le développement Python professionnel

PyCharm

Développé par JetBrains, PyCharm est un IDE puissant et très apprécié, particulièrement adapté aux projets Python de grande envergure et aux équipes professionnelles. Il existe en version gratuite (Community) et payante (Professional, avec des fonctionnalités avancées pour le web, la data science, etc.).

Points forts pour l'IT :

- **Analyse de code intelligente :** Détecte les erreurs, les problèmes de style (PEP 8) et les vulnérabilités potentielles.
- **Débogueur avancé :** Un des meilleurs débogueurs du marché pour Python, essentiel pour les applications complexes.
- **Intégration poussée :** Support natif pour les frameworks web (Django, Flask), les bases de données, Docker, Git.
- **Environnements virtuels :** Gestion fluide de `venv` et `conda`.

Installation : Téléchargez la version Community sur https://www.jetbrains.com/pycharm/download/. L'assistant d'installation est intuitif et vous guidera pour configurer votre interpréteur Python.

VS Code (Visual Studio Code) : La flexibilité au service du développeur Python

VS Code

Visual Studio Code de Microsoft est un éditeur de code open source, léger et multiplateforme. Sa force réside dans son écosystème d'extensions très riche qui lui permet de se transformer en un IDE complet pour Python (et bien d'autres langages). Il est extrêmement populaire pour sa rapidité, sa flexibilité et sa grande personnalisation.

Points forts pour l'IT :

- **Léger et rapide :** Démarrage instantané, faible consommation de ressources.
- **Extensions puissantes :** L'extension officielle "Python" de Microsoft (avec Pylance pour l'autocomplétion) offre une expérience IDE complète.
- **Polyvalence :** Idéal pour les scripts d'automatisation, le développement web (front-end/back-end), la gestion de configuration (DevOps), et l'analyse de logs.
- **Intégration Git :** Support natif et intuitif pour le contrôle de version.

Installation :

1. Téléchargez VS Code sur https://code.visualstudio.com/.
2. Installez l'extension officielle "Python" de Microsoft via le gestionnaire d'extensions intégré (accessible via Ctrl+Shift+X).

Jupyter Notebook / JupyterLab : L'environnement interactif pour la Data Science et l'analyse

Jupyter Notebook est un environnement de développement web interactif qui permet de créer des documents appelés "notebooks". Ces notebooks sont des documents "vivants" qui mélangent :

- Du code exécutable (Python, R, Julia).

- Du texte formaté (Markdown), des équations (LaTeX).
- Des visualisations (graphiques, tableaux) et des médias.

JupyterLab est une version plus avancée et modulable de Jupyter Notebook, offrant une interface plus complète avec des onglets, un explorateur de fichiers et la possibilité d'ouvrir différents types de documents.

Points forts pour l'IT (Data Science Recherche) :

- **Interactivité par cellule :** Exécutez le code cellule par cellule, parfait pour l'exploration de données, l'expérimentation et le prototypage rapide d'algorithmes.
- **Affichage Riche :** Affiche directement les graphiques (Matplotlib, Seaborn), les tableaux (Pandas) et les résultats de calculs inline, idéal pour l'analyse exploratoire des données.
- **Documentation Intégrée et Reproductible :** Combinez du code avec des explications textuelles en Markdown, créant des rapports techniques interactifs et reproductibles.
- **Partage Facile :** Les fichiers `.ipynb` sont faciles à partager et peuvent être ouverts et exécutés par d'autres utilisateurs via Jupyter ou des plateformes comme Google Colab.

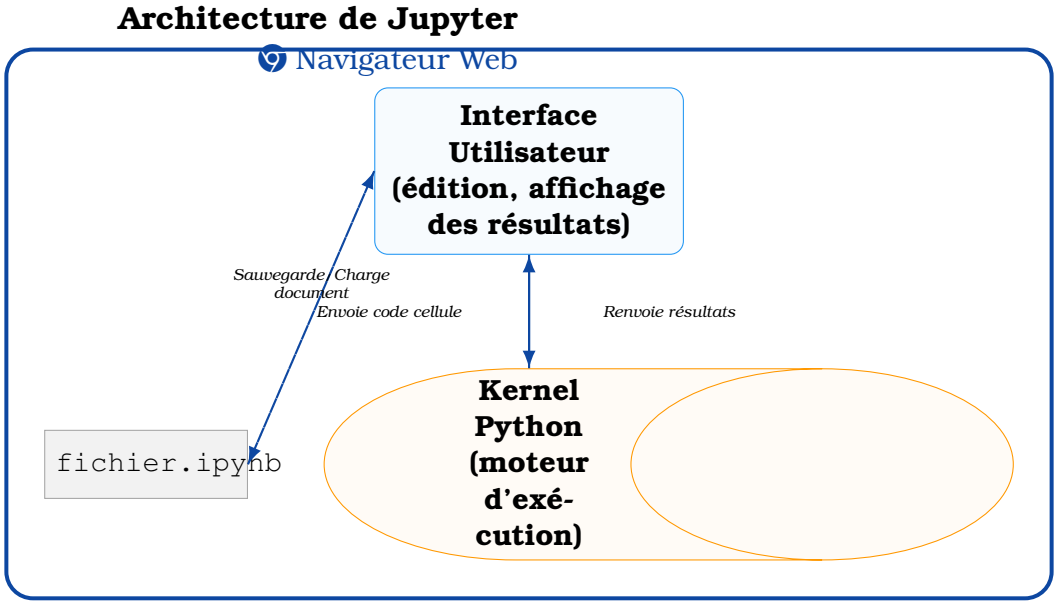


FIGURE 1.7 – *Fonctionnement interactif de Jupyter Notebook/Lab, idéal pour l'analyse de données et la recherche IT.*

Installation et Lancement :

- **Si Anaconda est installé :** Jupyter Notebook et JupyterLab sont généralement déjà inclus. Lancez-les depuis le "Anaconda Navigator" ou via votre terminal :

```
Terminal
jupyter notebook
# ou pour JupyterLab
jupyter lab
```

Si Python officiel est installé (avec pip) :

```
Terminal
pip install notebook
# ou pour l'interface plus riche de JupyterLab
pip install jupyterlab
```

Puis lancez avec `jupyter notebook` ou `jupyter lab`.

TABLE 1.3 – *Comparatif des environnements de développement Python pour les professionnels de l'IT*

Environnemen	Caractéristiques Principales	Scénarios d'Utilisation Idéaux en IT
REPL (Console)	Exécution immédiate, ligne par ligne, retour instantané.	Tests rapides de syntaxe/-fonctions, exploration d'APIs, debugging de snippets.
Jupyter Notebook / JupyterLab	Interactif, mélange code/texte/graphiques, exécution par cellule.	Analyse exploratoire de données, rapports d'audit interactifs, prototypage ML/IA, tutoriels.
PyCharm	IDE complet : débogueur avancé, refactoring, gestion de projet, intégration Git.	Développement d'applications web (Django/Flask), microservices, backend complexes, grands projets d'équipe.
VS Code	Éditeur léger et extensible : extensions riches, support multi-langages, terminal intégré.	Scripts d'automatisation, développement web (front/-back), gestion de configuration (DevOps), analyse de logs.
Spyder	Orienté scientifique : explorateur de variables, console IPython, éditeur Matplotlib.	Data Science traditionnelle, analyse statistique, simulations (ingénierie, finance), bioinformatique.
Google Colab	Notebooks basés sur le cloud : GPU/TPU gratuits, partage facile, pas d'installation locale.	Projets Deep Learning collaboratifs, prototypage rapide d'IA, apprentissage à distance sans infrastructure matérielle.

1.5 Structure d'un programme Python : Organiser votre premier code

Écrire du code ne se résume pas à taper des instructions ; il s'agit aussi de les organiser de manière logique pour qu'elles soient comprises par la machine ET par les humains. En IT, un programme bien structuré est la clé de la **maintenabilité** et de la **collaboration**.

1.5.1 L'Indentation : La règle d'or de Python

En Python, l'**indentation** n'est pas juste pour la beauté du code, c'est une **règle de syntaxe** ! Elle définit les blocs de code. Imaginez-la comme les paragraphes d'un texte : chaque nouveau paragraphe (bloc de code) commence avec une indentation.

IndentationError : L'erreur fréquente du débutant

Une `IndentationError` survient si votre code n'est pas indenté correctement (par exemple, un mélange d'espaces et de tabulations, ou une indentation incohérente). L'usage recommandé est de **4 espaces** par niveau d'indentation. Votre IDE configurera cela automatiquement.

In [0]:

```
## Un code simple pour illustrer l'indentation
    serveur_actif = True

    if serveur_actif: # Si le serveur est actif...
    print("Le serveur est opérationnel.") # ... alors exécute cette ligne (indentée)
    print("Vérification en cours...")     # ... et celle-ci (même indentation)
    print("Fin de la vérification.")      # Cette ligne s'exécute toujours, elle n'est pas indentée
```

Out[0]:

```
Le serveur est opérationnel. Vérification en cours... Fin de la vérification.
```

Utilité IT de l'indentation : L'indentation obligatoire rend le code Python naturellement lisible, réduisant les erreurs de logique et facilitant la compréhension mutuelle au sein d'une équipe IT.

1.5.2 Les commentaires : Votre code parle aux humains

Les commentaires sont des notes dans votre code destinées aux humains (vous, et vos collègues !). L'interpréteur Python les ignore. Ils sont essentiels pour expliquer ce que fait une partie complexe du code, pourquoi un choix a été fait, ou pour laisser une note importante.

In [0]:

```
## Ceci est un commentaire sur une seule ligne.
    # Il explique la ligne de code suivante.
```

```
3
4       # Déclaration d'une variable pour stocker le nom de
            l'application
5       nom_application = "GestionRH"
6
7       # Affichage d'un message de démarrage de l'application
8       print(f"Démarrage de l'application :
            {nom_application}")
9
10      # Les commentaires peuvent aussi être placés après une
            instruction
11      version = "1.0.0" # Numéro de version actuel
12      print(f"Version : {version}")
```

Out[0]:

```
Démarrage de l'application : GestionRH Version : 1.0.0
```

L'art du commentaire efficace en IT

- **Expliquez le "pourquoi", pas le "quoi" :** Le code explique souvent le "quoi". Le commentaire doit expliquer le "pourquoi" (ex : "Pourquoi utiliser cette technique plutôt qu'une autre ?").
- **Soyez concis et pertinent :** Des commentaires trop longs ou évidents sont contre-productifs.

Utilité IT : Les commentaires sont cruciaux pour la **maintenabilité** du code et le **partage des connaissances** dans les équipes IT. Ils aident les nouveaux membres à comprendre rapidement le projet.

1.5.3 Point d'entrée d'un script : Votre programme commence ici

Pour des scripts simples écrits dans un seul fichier, Python exécute le code de haut en bas. Cependant, une bonne pratique existe pour définir explicitement le "point de départ" de votre script. Ce n'est pas une obligation pour les scripts très simples, mais c'est une convention professionnelle que vous verrez souvent.

```
mon_premier_script.py

    # 1. Zone de définition (fonctions, variables globales, etc.)
    message_bienvenue = "Bienvenue dans votre premier script Python !"

    def afficher_infos_systeme():
    # Cette fonction sera implémentée plus tard pour afficher des infos système
    return "Informations système fictives."

    # 2. Point d'entrée principal du script
    # Ce bloc s'exécute UNIQUEMENT si le script est lancé directement
    if __name__ == "__main__":
    print(message_bienvenue)
    infos = afficher_infos_systeme()
    print(infos)
    print("Script terminé.")
```

Utilité IT de ce bloc (`if __name__=="__main__":`) :

- **Clarté structurelle :** Il indique clairement où commence l'exécution principale de votre script.
- **Flexibilité future :** Sans rentrer dans les détails techniques complexes de "modules" pour le moment, sachez que ce bloc permet à votre script d'être potentiellement réutilisé par d'autres programmes sans exécuter sa logique principale.

1.6 Variables et types de données de base : Les fondations de l'information IT

En programmation, manipuler l'information est au cœur de tout ce que vous ferez en IT. Pour cela, vous aurez besoin de deux concepts fondamentaux : les **variables** pour stocker cette information, et les **types de données** pour définir sa nature. Comprendre ces concepts est la première étape pour écrire des programmes qui gèrent des données de manière efficace et sans erreur.

1.6.1 Variables : Conteneurs pour vos données

Imaginez une variable comme une **boîte nommée** dans la mémoire de votre ordinateur. Vous pouvez y ranger une valeur, et cette valeur peut changer au cours de l'exécution de votre programme.

Variable en programmation

Une **variable** est un emplacement nommé dans la mémoire de l'ordinateur qui permet de stocker une valeur. Son contenu peut être modifié pendant l'exécution d'un programme.

Déclaration et affectation en Python

En Python, la déclaration (créer la boîte) et l'affectation (y mettre une valeur) se font en une seule étape simple, en utilisant l'opérateur `=`. Python est un langage à **typage dynamique** : vous n'avez pas besoin de spécifier le type de la variable (texte, nombre, etc.) à l'avance ; Python le déduit de la valeur que vous lui donnez.

In [0]:

```
## Affectation de différentes valeurs à des variables
    nombre_utilisateurs_actifs = 150 # Un nombre entier
    nom_application = "Gestion de Stock" # Une chaîne de
        caractères
    est_service_demarre = True # Une valeur booléenne
        (vrai/faux)

    # Affichage des valeurs des variables
    print("Utilisateurs actifs:",
        nombre_utilisateurs_actifs)
    print("Application courante:", nom_application)
    print("Service démarré:", est_service_demarre)

    # La valeur d'une variable peut être modifiée à tout
        moment
    nombre_utilisateurs_actifs = 155
    print("Nouvel affichage des utilisateurs:",
        nombre_utilisateurs_actifs)
```

Out[0]:

```
Utilisateurs actifs: 150 Application courante: Gestion de Stock
Service démarré: True Nouvel affichage des utilisateurs: 155
```

Règles et bonnes pratiques pour les noms de variables en IT

Choisir des noms de variables **clairs et descriptifs** est crucial pour la lisibilité et la maintenabilité de votre code, surtout quand vous travaillez en équipe sur des projets IT :

- **Descriptifs :** `nom_utilisateur` est bien plus explicite que `n` ou `x`.
- **Minuscules avec underscores (`snake_case`) :** C'est la convention Python (ex : `nombre_elements`, `chemin_fichier_log`).
- **Pas de caractères spéciaux ou d'espaces :** Utilisez _ à la place des espaces.
- **Ne pas commencer par un chiffre :** `1_variable` est invalide, mais `variable_1` est valide.
- **Éviter les mots-clés réservés de Python :** (ex : `print`, `if`, `for`, `class`).

1.6.2 Types de données de base : La nature de l'information

Le **type de données** d'une variable indique la catégorie de valeur qu'elle contient (nombre, texte, vrai/faux, etc.). Bien que Python détermine les types automatiquement, comprendre leur fonctionnement est essentiel pour :

- Choisir le bon type pour stocker vos données efficacement.
- Effectuer les opérations correctes sur ces données.
- Éviter les erreurs de type dans vos programmes.

Vous pouvez connaître le type de n'importe quelle variable ou valeur en utilisant la fonction `type()`.

Nombres : `int` (entiers) et `float` (flottants)

Les nombres sont le pain quotidien de tout programme IT, que ce soit pour compter des ressources, calculer des performances ou manipuler des données financières.

- **Entiers (`int`) :** Représentent des nombres entiers, positifs ou négatifs, sans partie décimale. **Usages IT :** Compteurs (nombre de requêtes), identifiants (ID utilisateur), ports réseau (80, 443), adresses IP (parties).
- **Flottants (`float`) :** Représentent des nombres réels, avec une partie décimale. **Usages IT :** Mesures précises (température CPU : 65.5°C), calculs de latence réseau (12.345 ms), taux et pourcentages (taux d'erreur : 0.0125).

In [0]:

```
## Exemples de nombres entiers et flottants
    nombre_machines_virtuelles = 50
    version_os = 10
    print(f"Type de 'nombre_machines_virtuelles':
        {type(nombre_machines_virtuelles)}")
    print(f"Type de 'version_os': {type(version_os)}")

    latence_moyenne_ms = 12.34
    taux_utilisation_cpu = 0.855 # 85.5%
    print(f"Type de 'latence_moyenne_ms':
        {type(latence_moyenne_ms)}")
    print(f"Type de 'taux_utilisation_cpu':
        {type(taux_utilisation_cpu)}")
```

Out[0]:

$$\texttt{Type de 'nombre}_{m}achines_{v}irtuelles' :< class'int' > Typede'version_{o}s' :< class'int' > Typede'latence_{m}oyenne_{m}s' :< class'float' > Typede'taux_{u}tilisation_{c}pu' :< class'float' >$$

Attention à la précision des flottants en IT

Les nombres flottants peuvent parfois avoir de très petites imprécisions dues à la manière dont les ordinateurs stockent les nombres à virgule. Pour la plupart des usages, c'est transparent, mais pour des calculs financiers critiques ou des comparaisons d'égalité strictes, soyez vigilant. Pour une précision décimale exacte, Python offre le module `decimal`.

Chaînes de caractères (`str`) : Manipuler le texte et les informations structurées

Le texte est omniprésent en IT. Les chaînes de caractères (`str`) sont utilisées pour stocker et manipuler toutes sortes d'informations textuelles.

- **Déclaration :** Les chaînes sont délimitées par des guillemets simples (`'`) ou doubles (`""`).
- **Usages IT :** Noms de fichiers, messages de logs, adresses IP, URLs, noms d'utilisateurs, mots de passe (avant hachage !), requêtes SQL.

In [0]:

```
## Déclaration de chaînes de caractères
```

```
    nom_fichier_log = "access_2024-03-15.log"
    message_alerte = 'Alerte sécurité: Tentative de
        connexion suspecte.'
    adresse_ip_source = "192.168.1.10"
    print(f"Type de 'nom_fichier_log':
        {type(nom_fichier_log)}")
    print(f"Adresse IP source: {adresse_ip_source}")

    # Concaténation de chaînes (assembler plusieurs
        chaînes)
    chemin_complet_log = "/var/log/apache/" +
        nom_fichier_log
    print(f"Chemin complet du log: {chemin_complet_log}")

    # Longueur d'une chaîne avec len()
    print(f"La longueur du message d'alerte est de
        {len(message_alerte)} caractères.")
```

Out[0]:

```
Type de 'nom_fichier_log': <class 'str'> Adresse
IP source: 192.168.1.10 Chemin complet du log:
/var/log/apache/access_2024-03-15.log La longueur du message
d'alerte est de 53 caractères.
```

Chaînes et sécurité IT

La manipulation de chaînes provenant d'utilisateurs est une source majeure de vulnérabilités. **Validez et nettoyez toujours les entrées utilisateur** (ex : pour éviter les injections SQL ou les attaques XSS). Ne stockez jamais de mots de passe en clair.

Booléens (`bool`) : La logique Vrai/Faux

Les booléens sont le fondement de la prise de décision en informatique. Une variable booléenne ne peut avoir que deux valeurs : `True` (vrai) ou `False` (faux).

- **Usages IT :** Représenter l'état d'un système (serveur actif ?), le résultat d'une condition (utilisateur authentifié ?), l'activation d'une fonctionnalité (mode maintenance activé ?).

In [0]:

```
## Exemples de variables booléennes
    serveur_principal_actif = True
    sauvegarde_terminee_avec_erreurs = False
    print(f"Type de 'serveur_principal_actif':
        {type(serveur_principal_actif)}")
    print(f"Sauvegarde réussie sans erreur: {not
        sauvegarde_terminee_avec_erreurs}") # Utilisation
        de 'not'

    # Les comparaisons renvoient des booléens
    charge_cpu = 95
    seuil_critique = 90
    alerte_cpu = charge_cpu > seuil_critique # La
        condition est-elle vraie ?
    print(f"Alerte CPU déclenchée: {alerte_cpu}")
```

Out[0]:

```
Type de 'serveur_principal_actif': <class 'bool'> Sauvegarde
réussie sans erreur: True Alerte CPU déclenchée: True
```

Logique binaire et informatique

L'informatique est basée sur la logique binaire (0 ou 1, True ou False). Comprendre les booléens est la première étape pour écrire des programmes capables de prendre des décisions complexes basées sur l'état de votre système IT.

1.6.3 Conversion de types (Type Casting) : Adapter les données

En IT, les données que vous recevez ne sont pas toujours dans le format idéal. La **conversion de type** (ou *type casting*) est l'opération qui consiste à transformer une valeur d'un type de données vers un autre. C'est indispensable pour manipuler des données provenant de saisies utilisateur, de fichiers ou de bases de données.

In [0]:

```
## 1. Conversion d'une chaîne de caractères en nombre (le
    plus fréquent)
```

```
    port_config_str = "443" # Port reçu d'un fichier de configuration (c'est du texte)
    port_config_int = int(port_config_str) # On le convertit en entier pour pouvoir faire des calculs ou des comparaisons
    print(f"Port en entier : {port_config_int}, type : {type(port_config_int)}")

    charge_cpu_str = "85.7" # Charge CPU lue d'un capteur (texte)
    charge_cpu_float = float(charge_cpu_str) # On le convertit en flottant
    print(f"Charge CPU en flottant : {charge_cpu_float}, type : {type(charge_cpu_float)}")

    # 2. Conversion d'un nombre en chaîne (pour affichage ou concaténation)
    id_transaction = 987654
    log_message = "Transaction ID: " + str(id_transaction) + " effectuée avec succès."
    print(log_message)

    # 3. Conversion en booléen
    # int en bool : 0 devient False, tout autre nombre devient True
    # str en bool : chaîne vide ("") devient False, toute autre chaîne devient True
    bool_zero = bool(0)
    bool_non_zero = bool(50)
    bool_empty_str = bool("")
    bool_non_empty_str = bool("OK")
    print(f"bool(0): {bool_zero}, bool(50): {bool_non_zero}, bool(''): {bool_empty_str}, bool('OK'): {bool_non_empty_str}")
```

Out[0]:

```
Port en entier : 443, type : <class 'int'> Charge CPU en flottant : 85.7, type : <class 'float'> Transaction ID: 987654 effectuée avec succès. bool(0): False, bool(50): True, bool(''): False, bool('OK'): True
```

Risque de `ValueError` lors de la conversion : L'importance du contrôle d'entrée

Une conversion de type échoue si la valeur ne peut pas être transformée dans le type cible. Par exemple, `int("bonjour")` provoquera une `ValueError` car "bonjour" n'est pas un nombre entier valide. En IT, c'est un point critique, surtout lors de la lecture de saisies utilisateur ou de données provenant de sources externes. La gestion de ces erreurs (via les blocs `try-except`) est essentielle pour rendre vos programmes robustes et tolérants aux pannes, et sera abordée dans un chapitre ultérieur.

1.7 Les Opérateurs en Python : Manipuler les données

Les opérateurs sont les outils qui permettent de **calculer**, **comparer** et **combiner** les données dans vos programmes. Maîtriser les opérateurs de base est essentiel pour écrire des programmes fonctionnels.

Opérateur

Un **opérateur** est un symbole qui effectue une opération sur une ou plusieurs valeurs appelées **opérandes** et produit un résultat.
Exemple : Dans `5 + 3`, le `+` est l'opérateur, `5` et `3` sont les opérandes, et `8` est le résultat.

1.7.1 Opérateurs Arithmétiques

Les opérateurs arithmétiques effectuent des **calculs mathématiques** sur des nombres.

TABLE 1.4 – *Opérateurs arithmétiques en Python*

Op.	Nom	Exemple	Résultat	Description
+	Addition	7 + 3	10	Somme
-	Soustraction	7 - 3	4	Différence
*	Multiplication	7 * 3	21	Produit
/	Division	7 / 2	3.5	Résultat décimal
//	Division entière	7 // 2	3	Partie entière
%	Modulo	7 % 3	1	Reste de la division
**	Puissance	2 ** 3	8	$2^3 = 8$

In [0]:

```
## Opérateurs arithmétiques de base
    a, b = 17, 5

    print(f"a = {a}, b = {b}")
    print(f"Addition       : a + b  = {a + b}")
    print(f"Soustraction   : a - b  = {a - b}")
    print(f"Multiplication: a * b  = {a * b}")
    print(f"Division       : a / b  = {a / b}")
    print(f"Div. entière   : a // b = {a // b}")
    print(f"Modulo         : a % b  = {a % b}")
    print(f"Puissance      : a ** 2 = {a ** 2}")
```

Out[0]:

```
a = 17, b = 5 Addition : a + b = 22 Soustraction : a - b = 12
Multiplication: a * b = 85 Division : a / b = 3.4 Div. entière :
a // b = 3 Modulo : a Puissance : a ** 2 = 289
```

Division entière et modulo

Ces deux opérateurs sont très utiles en programmation :

- `//` donne le **quotient** (nombre de fois que le diviseur "entre" dans le dividende)
- `%` donne le **reste** de la division

Exemple : $17 \div 5 = 3$ reste 2 donc `17 // 5 = 3` et `17 % 5 = 2`

Application : Tester si un nombre est pair : `nombre % 2 == 0`

1.7.2 Opérateurs de Comparaison

Les opérateurs de comparaison comparent deux valeurs et retournent un **booléen** (`True` ou `False`). Ils sont essentiels pour les conditions.

TABLE 1.5 – *Opérateurs de comparaison en Python*

Op.	Signification	Exemple	Résultat
==	Égal à	`5 == 5`	`True`
!=	Différent de	`5 != 3`	`True`
<	Inférieur à	`3 < 5`	`True`
>	Supérieur à	`5 > 3`	`True`
<=	Inférieur ou égal	`5 <= 5`	`True`
>=	Supérieur ou égal	`5 >= 3`	`True`

In [0]:

```
## Comparaisons de base
    x, y = 10, 20

    print(f"x = {x}, y = {y}")
    print(f"x == y : {x == y}")    # Égal ?
    print(f"x != y : {x != y}")    # Différent ?
    print(f"x < y  : {x < y}")     # Inférieur ?
    print(f"x > y  : {x > y}")     # Supérieur ?
    print(f"x <= y : {x <= y}")    # Inférieur ou égal ?
```

Out[0]:

```
x = 10, y = 20 x == y : False x != y : True x < y : True x > y :
False x <= y : True
```

⚠ Ne confondez pas = et ==

- = est l'**affectation** : `x = 5` (donne la valeur 5 à x)
- == est la **comparaison** : `x == 5` (vérifie si x vaut 5)

C'est une erreur très fréquente chez les débutants !

Comparaisons chaînées

Python permet d'écrire des comparaisons comme en mathématiques :

In [0]:

```
age = 25
        # Vérifier si age est entre 18 et 65
        print(18 <= age <= 65)   # Plus lisible que :
            age >= 18 and age <= 65
```

Out[0]:

```
True
```

1.7.3 Opérateurs d'Affectation

Les opérateurs d'affectation servent à **assigner** des valeurs aux variables. Les opérateurs **composés** combinent une opération avec l'affectation.

TABLE 1.6 – *Opérateurs d'affectation courants*

Op.	Description	Exemple	Équivalent à
=	Affectation simple	x = 5	–
+=	Ajouter et affecter	x += 3	x = x + 3
-=	Soustraire et affecter	x -= 3	x = x - 3
*=	Multiplier et affecter	x *= 2	x = x * 2
/=	Diviser et affecter	x /= 2	x = x / 2

In [0]:

```
## Opérateurs d'affectation composés
    compteur = 10
    print(f"Valeur initiale : {compteur}")

    compteur += 5    # Ajoute 5
    print(f"Après += 5 : {compteur}")

    compteur *= 2    # Multiplie par 2
    print(f"Après *= 2 : {compteur}")

    compteur -= 10   # Soustrait 10
    print(f"Après -= 10 : {compteur}")
```

Out [0] :

```
Valeur initiale : 10 Après += 5 : 15 Après *= 2 : 30 Après -= 10
: 20
```

Astuce Python : échange de variables

Python permet d'échanger deux variables sans variable temporaire :

In [0]:

```
a, b = 5, 10
print(f"Avant : a = {a}, b = {b}")

a, b = b, a  # Échange en une ligne !
print(f"Après : a = {a}, b = {b}")
```

Out [0] :

```
Avant : a = 5, b = 10 Après : a = 10, b = 5
```

1.7.4 Opérateurs Logiques

Les opérateurs logiques combinent des expressions booléennes. Ils sont indispensables pour créer des **conditions complexes**.

TABLE 1.7 – *Opérateurs logiques en Python*

Op.	Description	Exemple	Résultat
`and`	ET : vrai si les deux sont vrais	`True and False`	`False`
`or`	OU : vrai si au moins un est vrai	`True or False`	`True`
`not`	NON : inverse la valeur	`not True`	`False`

A	B	Résultat
V	V	**V**
V	F	**F**
F	V	**F**
F	F	**F**

A	B	Résultat
V	V	**V**
V	F	**V**
F	V	**V**
F	F	**F**

not

A	Résultat
V	**F**
F	**V**

FIGURE 1.8 – *Tables de vérité des opérateurs logiques (V = Vrai, F = Faux)*

In [0]:

```
## Opérateurs logiques
    age = 25
```

```
    a_permis = True

    # Peut conduire si majeur ET a le permis
    peut_conduire = age >= 18 and a_permis
    print(f"Peut conduire : {peut_conduire}")

    # Réduction si étudiant OU senior
    est_etudiant = True
    est_senior = False
    a_reduction = est_etudiant or est_senior
    print(f"A une réduction : {a_reduction}")

    # Accès refusé si NON authentifié
    est_authentifie = False
    acces_refuse = not est_authentifie
    print(f"Accès refusé : {acces_refuse}")
```

Out[0]:

```
Peut conduire : True A une réduction : True Accès refusé : True
```

1.7.5 Opérateurs d'Appartenance

Les opérateurs `in` et `not in` vérifient si une valeur est **présente dans une séquence** (liste, chaîne, etc.).

TABLE 1.8 – *Opérateurs d'appartenance*

Op.	Description	Exemple	Résultat
`in`	Est présent dans	`"a" in "chat"`	`True`
`not in`	N'est pas présent	`"z" not in "chat"`	`True`

In [0]:

```
## Vérifier la présence dans une liste
    fruits = ["pomme", "banane", "orange"]
    print(f"'pomme' dans fruits : {'pomme' in fruits}")
    print(f"'kiwi' dans fruits : {'kiwi' in fruits}")

    # Vérifier la présence dans une chaîne
    message = "Bonjour Python"
    print(f"'Python' dans message : {'Python' in message}")
    print(f"'Java' dans message : {'Java' in message}")
```

Out [0] :

```
'pomme' dans fruits : True 'kiwi' dans fruits : False 'Python'
dans message : True 'Java' dans message : False
```

1.7.6 Priorité des Opérateurs

Quand une expression contient plusieurs opérateurs, Python les évalue selon un **ordre de priorité** (comme en mathématiques).

TABLE 1.9 – *Priorité des opérateurs (du plus au moins prioritaire)*

Priorité	Opérateurs	Description
1 (haute)	`()`	Parenthèses
2	`**`	Puissance
3	`* / // %`	Multiplication, divisions, modulo
4	`+ -`	Addition, soustraction
5	`< > <= >= == !=`	Comparaisons
6	`not`	NON logique
7	`and`	ET logique
8 (basse)	`or`	OU logique

In [0] :

```
## Priorité des opérateurs
    resultat = 2 + 3 * 4       # * avant +
    print(f"2 + 3 * 4 = {resultat}")

    resultat = (2 + 3) * 4     # Parenthèses d'abord
    print(f"(2 + 3) * 4 = {resultat}")

    resultat = 2 ** 3 * 2      # ** avant *
    print(f"2 ** 3 * 2 = {resultat}")
```

Out [0] :

```
2 + 3 * 4 = 14 (2 + 3) * 4 = 20 2 ** 3 * 2 = 16
```

Conseil pratique

En cas de doute sur la priorité, utilisez des **parenthèses** ! Elles rendent votre code plus clair et évitent les erreurs.

```
# Difficile à lire
resultat = a + b * c / d - e
```

```
# Plus clair
resultat = a + ((b * c) / d) - e
```

1.8 Travaux Pratiques (TP) du Module 1

Ces travaux pratiques vous permettent d'appliquer les connaissances acquises dans ce module : installation, environnement, variables, types de données et opérateurs.

Travaux Pratiques : Mettre les mains dans le code

TP 1 – Vérification d'environnement et découverte du REPL

Objectif : Vérifier que votre environnement Python est fonctionnel et découvrir l'interpréteur interactif.

Partie A – Vérification de l'installation

1. Ouvrez un terminal (Invite de commandes sur Windows, Terminal sur macOS/Linux).
2. Exécutez les commandes suivantes et notez les résultats :
   - `python -version` (ou `python3 -version`)
   - `pip -version`
3. Si Python n'est pas reconnu, vérifiez qu'il est bien ajouté au PATH.

Partie B – Exploration du REPL

1. Lancez l'interpréteur Python en tapant `python` (ou `python3`).
2. Dans le REPL, effectuez les opérations suivantes :
   - Affichez `"Bonjour, Python !"`
   - Calculez `2025 - 1991` (âge de Python)
   - Calculez `2 ** 10` (valeur de 1 Ko en octets)
   - Testez `import this` pour afficher le Zen de Python
3. Quittez le REPL avec `exit()` ou `Ctrl+D`.

Livrable : Capture d'écran montrant les commandes et résultats.

TP 2 – Premier script : Variables et interactions utilisateur

Objectif : Créer un script Python structuré utilisant les variables, les types de données et les interactions utilisateur.

Instructions :

1. Ouvrez votre IDE.
2. Créez un fichier nommé `fiche_identite.py`.
3. Écrivez un programme qui :
   - Affiche un message de bienvenue
   - Demande le prénom de l'utilisateur (`input()`)
   - Demande son année de naissance
   - Demande sa ville
   - Calcule son âge approximatif (2024 - année)
   - Affiche une fiche récapitulative avec toutes les informations
4. Utilisez des **f-strings** pour le formatage.
5. Ajoutez des **commentaires** pour expliquer votre code.

Exemple de sortie attendue :

```
=== Bienvenue dans le générateur de fiche d'identité ===
Votre prénom : Avoundoupou
Votre année de naissance : 2005
Votre ville : Lomé

=== Votre fiche ===
Prénom : Alice
Âge approximatif : 20 ans
Ville : Lomé
```

Bonus : Affichez le type de chaque variable avec `type()`.
Livrable : Le fichier `fiche_identite.py` et une capture de son exécution.

TP 3 – Environnements virtuels et gestion des packages

Objectif : Maîtriser la création d'environnements virtuels et l'installation de packages.

Instructions :

1. Ouvrez un terminal et placez-vous dans un dossier de travail.
2. **Créez un environnement virtuel :**
   - Avec `venv` : `python -m venv mon_env`
   - Ou avec `conda` : `conda create -n mon_env python=3.11`
3. **Activez l'environnement :**
   - Windows : `mon_env\Scripts\activate`
   - macOS/Linux : `source mon_env/bin/activate`
   - Conda : `conda activate mon_env`
4. **Vérifiez l'activation :**
   - `python -c "import sys; print(sys.executable)"`
5. **Installez des packages :**
   - `pip install requests`
   - `pip list` pour voir les packages installés
6. **Exportez les dépendances :**
   - `pip freeze > requirements.txt`
7. **Désactivez l'environnement :**
   - `deactivate` (ou `conda deactivate`)

Livrable : Capture d'écran de toutes les étapes + le fichier `requirements.txt`.

TP 4 – Calculatrice IT : Opérateurs et conversions

Objectif : Utiliser les opérateurs Python pour des calculs pratiques en IT.

Instructions :
Créez un fichier `calculatrice_it.py` qui réalise les calculs suivants :

Partie A – Calculs de stockage

1. Demandez une taille de fichier en Go à l'utilisateur.
2. Convertissez et affichez cette taille en :
   - Mégaoctets (Mo) : $\times 1024$
   - Kilooctets (Ko) : $\times 1024^2$
   - Octets : $\times 1024^3$
3. Utilisez l'opérateur `**` pour les puissances.

Partie B – Calculs avec modulo

1. Demandez un nombre de secondes à l'utilisateur.
2. Convertissez en heures, minutes et secondes.
3. **Indice :** Utilisez `//` et `%`
4. Exemple : 3725 secondes = 1h 2min 5s

Partie C – Vérifications avec opérateurs logiques

1. Demandez un pourcentage d'utilisation CPU.
2. Affichez :
   - `True` si usage $\geq 80\%$ (alerte)
   - `True` si usage entre 50% et 80% (attention)
   - `True` si usage $< 50\%$ (normal)

Livrable : Le fichier `calculatrice_it.py` et des captures d'exécution.

TP 5 – Projet intégrateur : Analyseur de système

Objectif : Combiner tous les concepts du module dans un projet complet.
Contexte : Créer un outil de diagnostic système simplifié.

Créez le fichier `diagnostic_systeme.py` avec les éléments suivants :

1. **En-tête :** Commentaires (nom, date) + titre affiché
2. **Collecte via `input()` :** Nom machine, CPU (%), Mémoire (%), Disque total (Go), Disque utilisé (Go)
3. **Calculs :** Espace disponible (Go, Mo, Ko), pourcentage utilisation disque
4. **Alertes (opérateurs logiques) :**
   - `alerte_cpu` : True si CPU $\geq$ 90%
   - `alerte_memoire` : True si mémoire $\geq$ 85%
   - `alerte_disque` : True si disque $\geq$ 90%
   - `systeme_critique` : True si $\geq$ 2 alertes
5. **Rapport :** Affichage formaté avec f-strings + état (Sain/Attention/Critique)

Exemple de sortie attendue :

```
=== DIAGNOSTIC SYSTÈME v1.0 ===
Machine : SERVEUR-01
CPU : 75% | Mémoire : 82% | Disque : 65%
Espace disponible : 175 Go (179200 Mo)
Alertes : CPU=False | Mém=False | Disque=False
»> ÉTAT : SAIN «<
```

Bonus : Utilisez `if __name__ == "__main__":`
Livrable : Fichier Python + capture d'écran d'exécution.

1.9 Quiz du Module 1 : Testez vos connaissances

Vrai/Faux QCM Question ouverte

1 VRAI/FAUX

Python est un langage compilé, ce qui signifie que le code source est entièrement transformé en code machine avant l'exécution.

☐ Vrai ☐ Faux

2 QCM

Selon le Zen de Python, quel principe est fondamental pour écrire du bon code ?

A. Le code doit être le plus court possible, même si c'est illisible

B. La lisibilité compte et l'explicite est mieux que l'implicite

C. Les performances sont plus importantes que la clarté

D. Il faut toujours utiliser les fonctionnalités les plus avancées

3 VRAI/FAUX

La fonction `input()` retourne **toujours** une chaîne de caractères (`str`), même si l'utilisateur saisit un nombre.

☐ Vrai ☐ Faux

4 QCM

Quel est l'objectif principal d'un environnement virtuel en Python ?

A. Accélérer l'exécution du code Python

B. Protéger le code source contre les virus

C. Isoler les dépendances d'un projet pour éviter les conflits

D. Permettre l'exécution de code C++ dans Python

5 QCM

Quel est le type de la variable `x` après l'instruction `x = 3.14` ?

A. `int` **B.** `float` **C.** `str` **D.** `double`

6 VRAI/FAUX

En Python, l'opérateur `/` effectue toujours une division **entière** (sans décimales).

☐ **Vrai** ☐ **Faux**

7 QCM

Quelle est la valeur de l'expression `17 % 5` ?

A. 3 **B.** 2 **C.** 3.4 **D.** 12

8 OUVERTE

Citez trois domaines d'application où Python est largement utilisé dans le secteur IT.

1. _____

2. _____

3. _____

9 QCM

Quelle est la valeur de l'expression `True and False or True` ?

A. `True` **B.** `False` **C.** Erreur **D.** `None`

10 OUVERTE

Vous obtenez l'erreur : `ModuleNotFoundError: No module named 'requests'`
Expliquez en 2-3 étapes comment résoudre ce problème.

Étape 1 : _____

Étape 2 : _____

Étape 3 : _____

CHAPITRE II

STRUCTURES DE CONTRÔLE : LOGIQUE ET AUTOMATISATION

Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ Comprendre et utiliser les **expressions booléennes** pour la prise de décision.
- ✓ Implémenter des **branchements conditionnels** (`if`, `elif`, `else`).
- ✓ Automatiser des tâches avec les **boucles** `for` et `while`.
- ✓ Contrôler le flux d'exécution avec `break`, `continue` et `pass`.
- ✓ Imbriquer des structures de contrôle pour résoudre des problèmes complexes.

Les structures de contrôle sont le **cœur de la logique** de tout programme. Elles permettent de :

- **Prendre des décisions** : exécuter du code selon des conditions
- **Répéter des actions** : automatiser des tâches répétitives
- **Contrôler le flux** : gérer l'ordre d'exécution des instructions

Les 3 types de flux d'exécution



FIGURE 2.1 – *Les trois paradigmes de flux d'exécution en programmation*

2.1 Expressions Booléennes : La base de la logique

Avant d'aborder les structures conditionnelles, il est essentiel de maîtriser les **expressions booléennes** — des expressions qui s'évaluent à `True` ou `False`.

Expression booléenne

Une **expression booléenne** est une expression qui, une fois évaluée, produit une valeur de type `bool` : soit `True` (vrai), soit `False` (faux).
Ces expressions sont la base de toute **prise de décision** en programmation.

2.1.1 Construction d'expressions booléennes

TABLE 2.1 – *Opérateurs pour construire des expressions booléennes*

Catégorie	Opérateurs	Exemple	Résultat
Comparaison	`==, !=, <, >, <=, >=`	`5 > 3`	`True`
Logique	`and, or, not`	`True and False`	`False`
Appartenance	`in, not in`	`"a" in "chat"`	`True`
Identité	`is, is not`	`x is None`	`True/False`

In [0]:

```
## Exemples d'expressions booléennes
    age = 25
    salaire = 35000
    est_etudiant = True

    # Comparaisons simples
    print(f"age >= 18 : {age >= 18}")
    print(f"salaire > 30000 : {salaire > 30000}")

    # Combinaisons avec opérateurs logiques
    majeur_et_employe = age >= 18 and salaire > 0
    print(f"Majeur ET employé : {majeur_et_employe}")

    # Comparaisons chaînées
    dans_tranche = 18 <= age <= 65
    print(f"Entre 18 et 65 ans : {dans_tranche}")
```

Out[0]:

```
age >= 18 : True salaire > 30000 : True Majeur ET employé : True
Entre 18 et 65 ans : True
```

2.1.2 Priorité des opérateurs logiques

Ordre d'évaluation

Les opérateurs logiques ont un ordre de priorité :

1. `not` (le plus prioritaire)
2. `and`
3. `or` (le moins prioritaire)

Conseil : Utilisez des parenthèses pour clarifier vos intentions !

In [0]:

```
## Priorité des opérateurs
    resultat1 = True or False and False  # and avant or
    resultat2 = (True or False) and False  # Parenthèses
        changent l'ordre

    print(f"True or False and False = {resultat1}")
    print(f"(True or False) and False = {resultat2}")
```

Out[0] :

```
True or False and False = True (True or False) and False = False
```

2.2 Structures Conditionnelles : Prendre des décisions

Les structures conditionnelles permettent d'exécuter du code **uniquement si** certaines conditions sont remplies.

2.2.1 L'instruction `if` simple

Instruction `if`

L'instruction `if` exécute un bloc de code **si et seulement si** la condition est vraie (`True`).
Syntaxe :

```
if condition:
# Code exécuté si condition est True
instruction1
instruction2
```

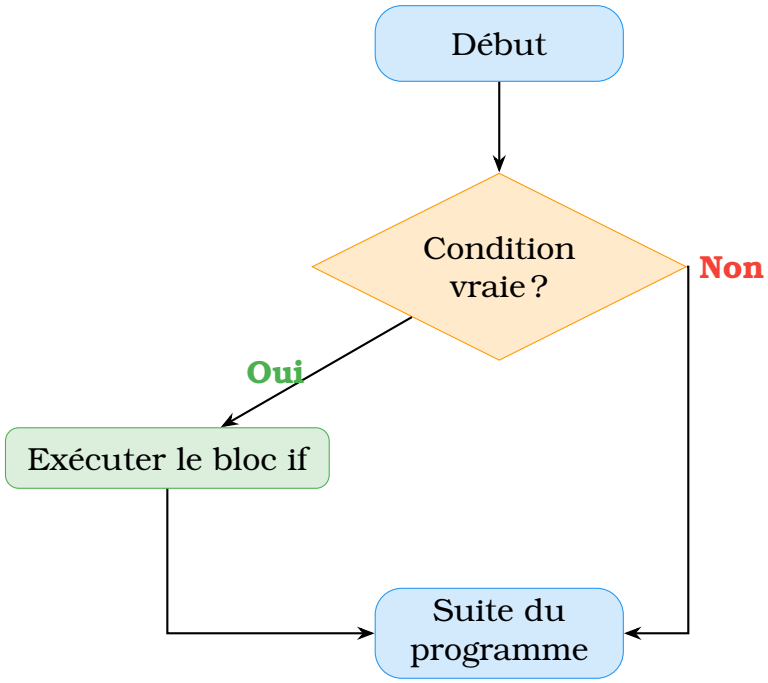


FIGURE 2.2 – *Organigramme de l'instruction `if` simple*

In [0]:

```
## Exemple : Vérification de l'âge
    age = 20

    if age >= 18:
    print("Vous êtes majeur.")
    print("Accès autorisé.")

    print("Fin du programme.")   # Toujours exécuté
```

Out[0]:

```
Vous êtes majeur. Accès autorisé. Fin du programme.
```

L'indentation est obligatoire !

En Python, le bloc de code sous le `if` **doit être indenté** (4 espaces recommandés). L'indentation définit ce qui appartient au bloc conditionnel.

```
if condition:
print("Dans le if")     # Indenté = fait partie du if
print("Hors du if")         # Non indenté = toujours exécuté
```

2.2.2 L'instruction `if-else`

Instruction `if-else`

L'instruction `else` définit un bloc de code à exécuter **si la condition est fausse**.
Syntaxe :

```
if condition:
# Code si True
else:
# Code si False
```

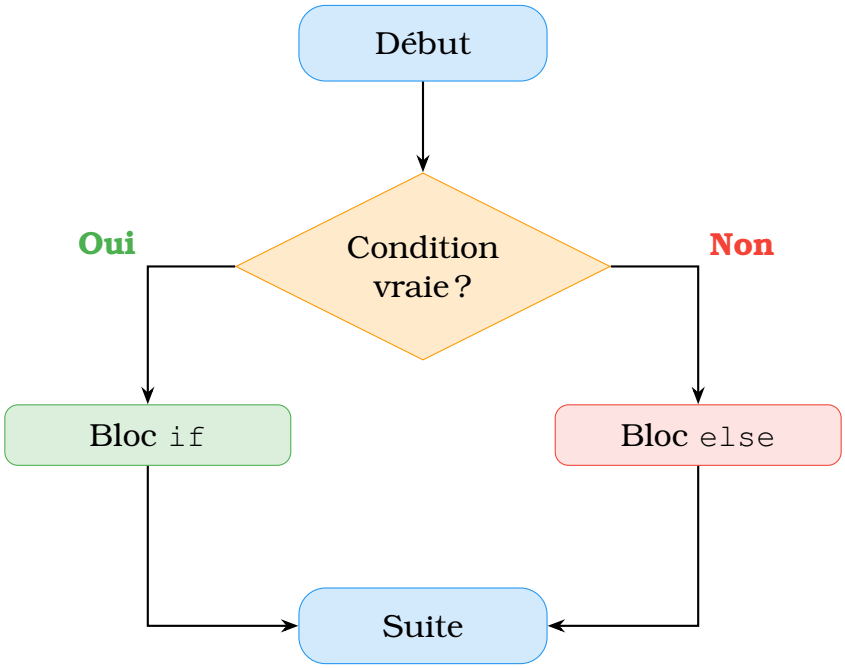


FIGURE 2.3 – *Organigramme de l'instruction* `if-else`

In [0]:

```
## Exemple : Pair ou impair
    nombre = 7

    if nombre % 2 == 0:
    print(f"{nombre} est pair.")
    else:
    print(f"{nombre} est impair.")
```

Out[0]:

```
7 est impair.
```

2.2.3 L'instruction `if-elif-else`

Pour gérer **plusieurs conditions mutuellement exclusives**, on utilise `elif` (contraction de "else if").

Instruction `elif`

`elif` permet de tester plusieurs conditions en séquence. La première condition vraie déclenche l'exécution de son bloc, puis on sort de la structure.

Syntaxe :

```
if condition1:
```

```
# Code si condition1 est True
elif condition2:
# Code si condition2 est True
elif condition3:
# Code si condition3 est True
else:
# Code si aucune condition n'est True
```

In [0]:

```
## Exemple : Classification de notes
    note = 75

    if note >= 90:
    mention = "Excellent"
    elif note >= 80:
    mention = "Très bien"
    elif note >= 70:
    mention = "Bien"
    elif note >= 60:
    mention = "Assez bien"
    elif note >= 50:
    mention = "Passable"
    else:
    mention = "Insuffisant"

    print(f"Note : {note}/100 - Mention : {mention}")
```

Out[0]:

```
Note : 75/100 - Mention : Bien
```

Ordre des conditions

Avec `elif`, l'ordre des conditions est **important** ! Python évalue les conditions de haut en bas et s'arrête à la première qui est vraie.
Conseil : Placez les conditions les plus spécifiques en premier.

2.2.4 Conditions imbriquées

On peut **imbriquer** des structures conditionnelles pour des logiques plus complexes.

In [0]:

```
## Exemple : Système de tarification
    age = 25
    est_etudiant = True

    if age < 18:
    tarif = "Junior"
    prix = 5
    elif age >= 65:
    tarif = "Senior"
    prix = 7
    else:
    # Adulte : vérifier si étudiant
    if est_etudiant:
    tarif = "Étudiant"
    prix = 8
    else:
    tarif = "Adulte"
    prix = 12

    print(f"Tarif : {tarif} - Prix : {prix}    ")
```

Out[0]:

```
Tarif : Étudiant - Prix : 8€
```

2.2.5 L'opérateur ternaire

Pour des conditions simples, Python offre une syntaxe condensée sur une ligne.

In [0]:

```
## Opérateur ternaire
    age = 20
    statut = "Majeur" if age >= 18 else "Mineur"
    print(statut)

    # Équivalent à :
    # if age >= 18:
    #     statut = "Majeur"
    # else:
    #     statut = "Mineur"
```

Out [0] :

```
Majeur
```

2.3 La Boucle `for` : Itérer sur des séquences

La boucle `for` permet de **parcourir** les éléments d'une séquence (liste, chaîne, range, etc.) et d'exécuter un bloc de code pour chaque élément.

Boucle `for`

La boucle `for` itère sur chaque élément d'un **itérable** (objet parcourable).
Syntaxe :

```
for variable in iterable:
# Code exécuté pour chaque élément
instruction
```

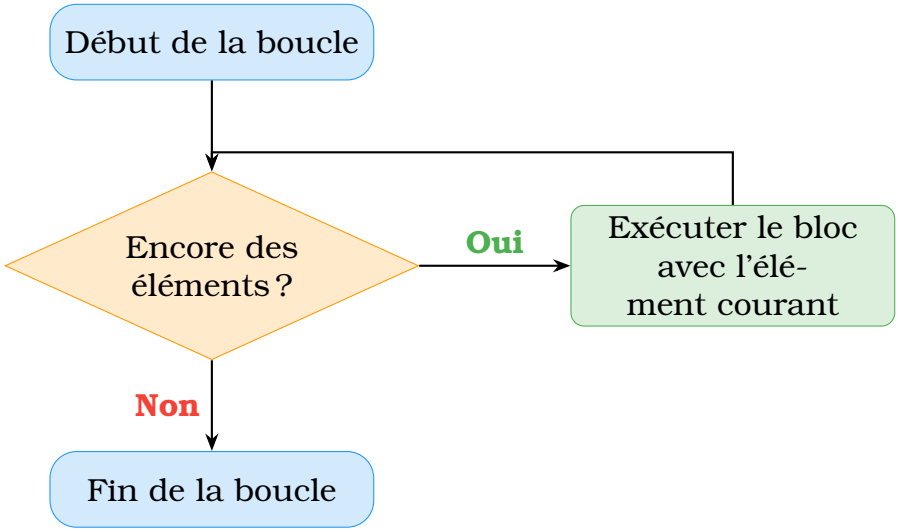


FIGURE 2.4 – *Organigramme de la boucle `for`*

2.3.1 Parcourir une liste

In [0] :

```
## Parcourir une liste de fruits
    fruits = ["pomme", "banane", "orange", "kiwi"]

    for fruit in fruits:
    print(f"J'aime les {fruit}s !")
```

Out[0]:

```
J'aime les pommes ! J'aime les bananes ! J'aime les oranges !
J'aime les kiwis !
```

2.3.2 La fonction `range()`

La fonction `range()` génère une séquence de nombres, très utile pour les boucles avec compteur.

TABLE 2.2 – *Variantes de* `range()`

Syntaxe	Description	Exemple
`range(n)`	0 à n-1	`range(5)` → 0,1,2,3,4
`range(a, b)`	a à b-1	`range(2,6)` → 2,3,4,5
`range(a, b, pas)`	a à b-1, par pas	`range(0,10,2)` → 0,2,4,6,8

In [0]:

```
## Utilisation de range()
    print("Comptage de 0 à 4 :")
    for i in range(5):
    print(i, end=" ")

    print("\n\nComptage de 1 à 5 :")
    for i in range(1, 6):
    print(i, end=" ")

    print("\n\nNombres pairs de 0 à 10 :")
    for i in range(0, 11, 2):
    print(i, end=" ")

    print("\n\nCompte à rebours :")
    for i in range(5, 0, -1):
    print(i, end=" ")
    print("Décollage !")
```

Out[0]:

```
Comptage de 0 à 4 : 0 1 2 3 4
Comptage de 1 à 5 : 1 2 3 4 5
Nombres pairs de 0 à 10 : 0 2 4 6 8 10
Compte à rebours : 5 4 3 2 1 Décollage !
```

2.3.3 Parcourir avec index : `enumerate()`

Pour obtenir à la fois l'index et la valeur, utilisez `enumerate()`.

In [0]:

```
## enumerate() : index + valeur
    langages = ["Python", "Java", "C++", "JavaScript"]

    for index, langage in enumerate(langages):
    print(f"{index + 1}. {langage}")
```

Out[0]:

```
1. Python 2. Java 3. C++ 4. JavaScript
```

2.3.4 Parcourir une chaîne de caractères

In [0]:

```
## Parcourir chaque caractère d'une chaîne
    mot = "Python"

    for lettre in mot:
    print(lettre, end="-")
```

Out[0]:

```
P-y-t-h-o-n-
```

2.3.5 Boucles `for` imbriquées

In [0]:

```
## Table de multiplication
    for i in range(1, 4):
    for j in range(1, 4):
    print(f"{i} x {j} = {i*j}", end="  |  ")
    print()  # Nouvelle ligne
```

Out[0]:

```
1 x 1 = 1 | 1 x 2 = 2 | 1 x 3 = 3 | 2 x 1 = 2 | 2 x 2 = 4 | 2 x
3 = 6 | 3 x 1 = 3 | 3 x 2 = 6 | 3 x 3 = 9 |
```

2.4 La Boucle `while` : Répéter tant que...

La boucle `while` exécute un bloc de code **tant qu'une condition est vraie**.

Boucle `while`

La boucle `while` répète un bloc de code tant que la condition reste `True`.
Syntaxe :

```
while condition:
# Code répété tant que condition est True
instruction
# Modifier quelque chose pour éviter boucle infinie !
```

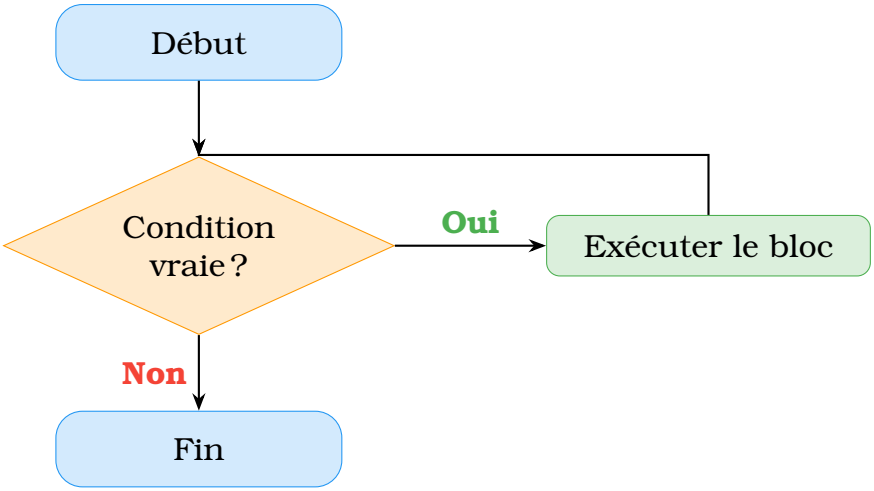


FIGURE 2.5 – *Organigramme de la boucle* `while`

In [0]:

```
## Exemple simple : compteur
    compteur = 1

    while compteur <= 5:
    print(f"Tour n°{compteur}")
    compteur += 1  # IMPORTANT : modifier le compteur !

    print("Boucle terminée.")
```

Out[0]:

```
Tour n°1 Tour n°2 Tour n°3 Tour n°4 Tour n°5 Boucle terminée.
```

Attention aux boucles infinies !

Si la condition ne devient jamais `False`, la boucle continue indéfiniment !
Toujours s'assurer que :

- La condition peut devenir fausse
- Une variable est modifiée dans la boucle

Pour arrêter une boucle infinie : `Ctrl+C`

2.4.1 Boucle `while` avec saisie utilisateur

In [0]:

```
## Demander un mot de passe jusqu'à réussite
    mot_de_passe_correct = "python123"
    tentatives = 0
    max_tentatives = 3

    while tentatives < max_tentatives:
    saisie = input("Mot de passe : ")
    tentatives += 1

    if saisie == mot_de_passe_correct:
    print("Accès autorisé !")
    break  # Sortir de la boucle
    else:
    restantes = max_tentatives - tentatives
    if restantes > 0:
    print(f"Incorrect. {restantes} tentative(s)
       restante(s).")
    else:
    print("Trop de tentatives. Compte bloqué.")
```

2.4.2 Différence entre `for` et `while`

TABLE 2.3 – *Quand utiliser `for` ou `while` ?*

Critère	`for`	`while`
Nombre d'itérations	Connu à l'avance	Inconnu à l'avance
Usage typique	Parcourir une séquence	Attendre une condition
Exemple	Liste, range, chaîne	Saisie utilisateur, jeux
Risque de boucle infinie	Faible	Élevé si mal conçu

2.5 Contrôle du Flux : `break`, `continue`, `pass`

Python offre trois instructions pour contrôler le comportement des boucles.

2.5.1 L'instruction `break`

`break` **termine immédiatement** la boucle en cours.

In [0]:

```
## Trouver le premier nombre divisible par 7
    for nombre in range(1, 100):
    if nombre % 7 == 0:
    print(f"Premier multiple de 7 : {nombre}")
    break  # Sortie immédiate
```

Out[0]:

```
Premier multiple de 7 : 7
```

2.5.2 L'instruction `continue`

`continue` **passe à l'itération suivante**, en ignorant le reste du bloc.

In [0]:

```
## Afficher uniquement les nombres impairs
    for i in range(1, 11):
    if i % 2 == 0:
    continue  # Sauter les pairs
    print(i, end=" ")
```

Out[0]:

```
1 3 5 7 9
```

2.5.3 L'instruction `pass`

`pass` est une instruction **vide** — elle ne fait rien. Utile comme placeholder.

In [0]:

```
## Structure à compléter plus tard
    for i in range(5):
    if i == 3:
    pass  # TODO: traitement spécial
    else:
    print(i)
```

Out[0]:

```
0 1 2 4
```

2.5.4 Schéma récapitulatif

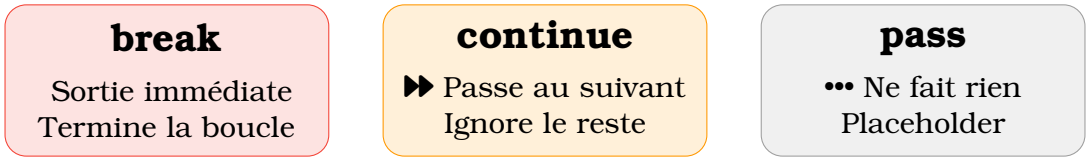


FIGURE 2.6 – *Les trois instructions de contrôle de flux*

2.6 Cas Pratiques IT

2.6.1 Validation de données

In [0]:

```
## Validation d'une adresse email simple
    email = "utilisateur@domaine.com"

    if "@" in email and "." in email:
    parties = email.split("@")
    if len(parties) == 2 and len(parties[0]) > 0 and
       len(parties[1]) > 3:
    print(f"Email valide : {email}")
    else:
    print("Format d'email incorrect")
    else:
    print("Email invalide : manque @ ou .")
```

Out[0]:

```
Email valide : utilisateur@domaine.com
```

2.6.2 Calcul de statistiques

In [0]:

```
## Calculer la moyenne, min et max d'une liste
    notes = [85, 92, 78, 95, 88, 76, 90]

    somme = 0
    minimum = notes[0]
```

```
    maximum = notes[0]

    for note in notes:
    somme += note
    if note < minimum:
    minimum = note
    if note > maximum:
    maximum = note

    moyenne = somme / len(notes)

    print(f"Notes : {notes}")
    print(f"Moyenne : {moyenne:.2f}")
    print(f"Minimum : {minimum}")
    print(f"Maximum : {maximum}")
```

Out[0]:

```
Notes : [85, 92, 78, 95, 88, 76, 90] Moyenne : 86.29 Minimum :
76 Maximum : 95
```

2.6.3 Recherche dans une liste

In [0]:

```
## Rechercher un utilisateur par ID
    utilisateurs = [
    {"id": 101, "nom": "Alice", "role": "admin"},
    {"id": 102, "nom": "Bob", "role": "user"},
    {"id": 103, "nom": "Charlie", "role": "user"},
    ]

    id_recherche = 102
    trouve = False

    for user in utilisateurs:
    if user["id"] == id_recherche:
    print(f"Utilisateur trouvé : {user['nom']}
        ({user['role']})")
    trouve = True
    break

    if not trouve:
```

```
18      print(f"Aucun utilisateur avec l'ID {id_recherche}")
```

```
Out[0]:
Utilisateur trouvé : Bob (user)
```

Récapitulatif : Structures de Contrôle

Conditions

- `if` : Si condition vraie
- `elif` : Sinon si...
- `else` : Sinon (défaut)
- Ternaire : `a if cond else b`

Boucle `for`

- Parcourt un itérable
- `range()` pour compteur
- `enumerate()` pour index
- Nombre d'itérations connu

Boucle `while`

- Tant que condition vraie
- Nombre d'itérations inconnu
- Attention boucles infinies !

Contrôle de flux

- `break` : Sortie immédiate
- `continue` : Itération suivante
- `pass` : Ne fait rien

2.7 Travaux Pratiques du Module 2

Travaux Pratiques : Structures de Contrôle

TP 1 – Calculatrice de mentions

Créez un programme qui :

1. Demande une note sur 100
2. Affiche la mention correspondante :
   - 90+ : Excellent | 80-89 : Très bien | 70-79 : Bien
   - 60-69 : Assez bien | 50-59 : Passable | <50 : Échec
3. Indique si l'étudiant est admis (note $\geq$ 50)

TP 2 – Table de multiplication

Créez un programme qui :

1. Demande un nombre à l'utilisateur
2. Affiche sa table de multiplication de 1 à 10
3. Utilise une boucle `for` avec `range()`

Bonus : Permettre à l'utilisateur de choisir la plage (ex : de 5 à 15).

TP 3 – Jeu du nombre mystère

Créez un jeu où :

1. Le programme choisit un nombre entre 1 et 100 (utilisez `random.randint()`)
2. L'utilisateur doit deviner le nombre
3. À chaque tentative, indiquer "Plus grand" ou "Plus petit"
4. Compter le nombre de tentatives
5. Utiliser une boucle `while`

TP 4 – Analyse de texte

Créez un programme qui analyse une phrase :

1. Demande une phrase à l'utilisateur
2. Compte le nombre de voyelles et de consonnes
3. Affiche le nombre de mots
4. Indique si la phrase contient des chiffres

★ TP 5 – Générateur de mot de passe

Créez un générateur de mot de passe :

1. Demande la longueur souhaitée (minimum 8)
2. Demande si inclure : majuscules, chiffres, caractères spéciaux
3. Génère un mot de passe aléatoire selon les critères
4. Vérifie la robustesse (faible/moyen/fort)

Modules utiles : `random`, `string`

2.8 Quiz du Module 2

1 **[Vrai/Faux]** L'instruction `elif` peut être utilisée sans `if` préalable.

☐ Vrai ☐ Faux

2 **[QCM]** Que produit `range(2, 8, 2)` ?

A. 2, 4, 6, 8

B. 2, 4, 6

C. 2, 3, 4, 5, 6, 7

D. 0, 2, 4, 6

3 **[Vrai/Faux]** Une boucle `while True:` s'exécute indéfiniment sauf si elle contient un `break`.

☐ Vrai ☐ Faux

4 **[QCM]** Quelle instruction permet de passer à l'itération suivante sans exécuter le reste du bloc ?

A. `break`

B. `continue`

C. `pass`

D. `next`

5 **[QCM]** Combien de fois s'exécute cette boucle ? `for i in range(5): print(i)`

A. 4 fois

B. 5 fois

C. 6 fois

D. Erreur

6 **[Vrai/Faux]** L'instruction `pass` termine la boucle en cours.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle est la valeur de `x` après : `x = 10 if 5 > 3 else 20` ?

A. 10

B. 20

C. 5

D. Erreur

8 **[Question ouverte]** Quelle est la différence principale entre `for` et `while` ?

9 **[QCM]** Que fait `enumerate()` ?

A. Compte les éléments

B. Retourne index et valeur

C. Trie la liste

D. Inverse la liste

10 **[Question ouverte]** Écrivez une boucle `for` qui affiche les nombres de 1 à 5.

CHAPITRE III

FONCTIONS EN PYTHON : MODULARITÉ ET RÉUTILISABILITÉ

Fonctions Python

Organiser, réutiliser et structurer votre code

Définition • Paramètres • Retour • Portée • Modules

Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ **Définir et appeler** des fonctions pour organiser votre code.
- ✓ Maîtriser les **paramètres** : positionnels, nommés, par défaut, `*args`, `**kwargs`.
- ✓ Utiliser le **retour de valeurs** avec `return`.
- ✓ Comprendre la **portée des variables** (locale, globale, `nonlocal`).
- ✓ Exploiter les **fonctions intégrées** et les **modules standards**.
- ✓ Documenter vos fonctions avec les **docstrings**.

Les fonctions sont des **blocs de code réutilisables** qui permettent de :

- **Éviter la répétition** : écrire une fois, utiliser partout
- **Organiser le code** : diviser un problème complexe en sous-problèmes
- **Faciliter la maintenance** : modifier à un seul endroit
- **Améliorer la lisibilité** : donner des noms explicites aux actions

Anatomie d'une fonction

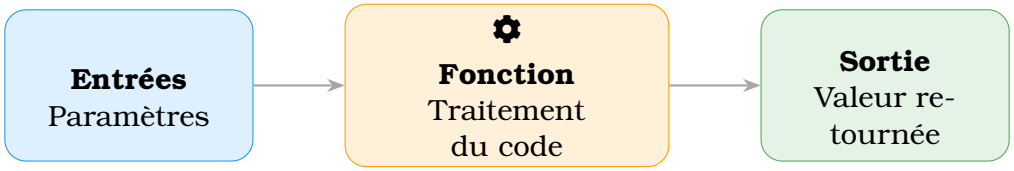


FIGURE 3.1 – *Le principe entrée → traitement → sortie d'une fonction*

3.1 Définition et Appel de Fonctions

3.1.1 Syntaxe de base

Fonction en Python

Une **fonction** est un bloc de code nommé qui effectue une tâche spécifique. Elle est définie avec le mot-clé `def` et peut être appelée (exécutée) autant de fois que nécessaire.

Syntaxe :

```
def nom_de_la_fonction(paramètres):
"""Docstring : description de la fonction"""
# Corps de la fonction
instructions
return valeur  # Optionnel
```

In [0]:

```
## Définition d'une fonction simple
    def saluer():
    """Affiche un message de salutation."""
    print("Bonjour et bienvenue !")

    # Appel de la fonction
    saluer()
    saluer()  # On peut l'appeler plusieurs fois
```

Out[0]:

```
Bonjour et bienvenue ! Bonjour et bienvenue !
```

Conventions de nommage

- Utilisez le `snake_case` : `calculer_moyenne`, `afficher_resultat`
- Choisissez des noms **explicites** décrivant l'action : `envoyer_email`, `valider_formulaire`
- Commencez par un **verbe** : `get_`, `set_`, `is_`, `calculate_`

3.1.2 Fonctions avec paramètres

Les **paramètres** permettent de passer des données à la fonction.

In [0]:

```
## Fonction avec un paramètre
    def saluer_personne(nom):
    """Salue une personne par son nom."""
    print(f"Bonjour, {nom} !")

    # Appel avec un argument
    saluer_personne("Alice")
    saluer_personne("Bob")
```

Out[0]:

```
Bonjour, Alice ! Bonjour, Bob !
```

In [0]:

```
## Fonction avec plusieurs paramètres
    def presenter(prenom, nom, age):
    """Présente une personne."""
    print(f"Je m'appelle {prenom} {nom} et j'ai {age}
        ans.")

    # Appel avec plusieurs arguments
    presenter("Marie", "Dupont", 25)
```

Out[0]:

```
Je m'appelle Marie Dupont et j'ai 25 ans.
```

Paramètre vs Argument

- **Paramètre** : variable dans la définition de la fonction (`nom`)
- **Argument** : valeur passée lors de l'appel (`"Alice"`)

3.2 Retour de Valeurs avec `return`

L'instruction `return` permet à une fonction de **renvoyer une valeur** à l'appelant.

Instruction `return`

`return` termine l'exécution de la fonction et renvoie une valeur. Sans `return` explicite, une fonction retourne `None`.

In [0]:

```
## Fonction qui retourne une valeur
    def additionner(a, b):
    """Retourne la somme de a et b."""
    resultat = a + b
    return resultat

    # Utiliser la valeur retournée
    somme = additionner(5, 3)
    print(f"5 + 3 = {somme}")

    # Utilisation directe
    print(f"10 + 20 = {additionner(10, 20)}")
```

Out[0]:

```
5 + 3 = 8 10 + 20 = 30
```

3.2.1 Retourner plusieurs valeurs

Python permet de retourner **plusieurs valeurs** sous forme de tuple.

In [0]:

```
## Retourner plusieurs valeurs
    def diviser(a, b):
    """Retourne le quotient et le reste de a / b."""
    quotient = a // b
```

```
    reste = a % b
    return quotient, reste  # Retourne un tuple

    # Récupérer les valeurs
    q, r = diviser(17, 5)
    print(f"17    5 = {q} reste {r}")

    # Ou récupérer le tuple
    resultat = diviser(17, 5)
    print(f"Résultat : {resultat}")
```

Out[0]:

```
17 ÷ 5 = 3 reste 2 Résultat : (3, 2)
```

3.2.2 Différence entre `return` et `print`

return ≠ print

- `print()` **affiche** à l'écran — ne renvoie rien (None)
- `return` **renvoie** une valeur — peut être stockée, réutilisée

In [0]:

```
## Fonction avec print (pas de return)
    def carre_print(x):
    print(x ** 2)

    # Fonction avec return
    def carre_return(x):
    return x ** 2

    # Différence
    resultat1 = carre_print(5)    # Affiche 25
    resultat2 = carre_return(5)   # Ne affiche rien, mais
       retourne 25

    print(f"resultat1 = {resultat1}")   # None !
    print(f"resultat2 = {resultat2}")   # 25

    # On peut réutiliser resultat2
    double = resultat2 * 2
```

```
    print(f"Double du carré : {double}")
```

Out[0]:

```
25 resultat1 = None resultat2 = 25 Double du carré : 50
```

3.3 Types de Paramètres

Python offre une grande flexibilité dans la gestion des paramètres.

3.3.1 Paramètres par défaut

On peut définir des **valeurs par défaut** pour les paramètres.

In [0]:

```
## Paramètres avec valeurs par défaut
def saluer(nom, message="Bonjour"):
    """Salue avec un message personnalisable."""
    print(f"{message}, {nom} !")

    # Appels
    saluer("Alice")                    # Utilise la valeur par défaut
    saluer("Bob", "Bonsoir")           # Remplace la valeur par défaut
    saluer("Charlie", message="Salut") # Argument nommé
```

Out[0]:

```
Bonjour, Alice ! Bonsoir, Bob ! Salut, Charlie !
```

Ordre des paramètres

Les paramètres avec valeurs par défaut doivent être **après** les paramètres obligatoires.

```
def fonction(obligatoire, optionnel="valeur"):  # OK
def fonction(optionnel="valeur", obligatoire):  # ERREUR !
```

3.3.2 Arguments positionnels vs nommés

In [0]:

```python
def creer_profil(nom, age, ville):
    """Crée un profil utilisateur."""
    print(f"Nom: {nom}, Age: {age}, Ville: {ville}")

    # Arguments positionnels (ordre compte)
    creer_profil("Alice", 25, "Paris")

    # Arguments nommés (ordre ne compte plus)
    creer_profil(ville="Lyon", nom="Bob", age=30)

    # Mélange (positionnels d'abord !)
    creer_profil("Charlie", ville="Marseille", age=28)
```

Out[0]:

```
Nom: Alice, Age: 25, Ville: Paris Nom: Bob, Age: 30, Ville: Lyon
Nom: Charlie, Age: 28, Ville: Marseille
```

3.3.3 Nombre variable d'arguments : `*args`

`*args` permet d'accepter un **nombre variable** d'arguments positionnels.

In [0]:

```python
## *args : nombre variable d'arguments
    def calculer_somme(*nombres):
    """Calcule la somme de tous les nombres."""
    total = 0
    for n in nombres:
    total += n
    return total

    # Appels avec différents nombres d'arguments
    print(calculer_somme(1, 2))
    print(calculer_somme(1, 2, 3, 4, 5))
    print(calculer_somme(10, 20, 30, 40, 50, 60))
```

Out[0]:

```
3 15 210
```

3.3.4 Arguments nommés variables : **kwargs

kwargs permet d'accepter un nombre variable d'arguments **nommés.

In [0]:

```python
## **kwargs : arguments nommés variables
    def afficher_infos(**infos):
    """Affiche toutes les informations passées."""
    for cle, valeur in infos.items():
    print(f"{cle}: {valeur}")

    # Appel avec différents arguments nommés
    afficher_infos(nom="Alice", age=25, ville="Paris")
    print("---")
    afficher_infos(produit="Laptop", prix=999, stock=50)
```

Out[0]:

```
nom: Alice age: 25 ville: Paris -- produit: Laptop prix: 999
stock: 50
```

3.3.5 Combinaison de tous les types

In [0]:

```python
## Combinaison complète
    def fonction_complete(obligatoire, optionnel="défaut",
        *args, **kwargs):
    """Démontre tous les types de paramètres."""
    print(f"Obligatoire: {obligatoire}")
    print(f"Optionnel: {optionnel}")
    print(f"Args: {args}")
    print(f"Kwargs: {kwargs}")

    fonction_complete("A", "B", 1, 2, 3, x=10, y=20)
```

Out[0]:

```
Obligatoire: A Optionnel: B Args: (1, 2, 3) Kwargs: 'x': 10,
'y': 20
```



FIGURE 3.2 – *Les quatre types de paramètres en Python*

3.4 Portée des Variables (Scope)

La **portée** d'une variable détermine où elle est accessible dans le code.

Portée (Scope)

- **Locale** : variable définie dans une fonction, accessible uniquement dans cette fonction
- **Globale** : variable définie au niveau du module, accessible partout
- **Englobante** : variable d'une fonction externe (fonctions imbriquées)

3.4.1 Variables locales

In [0]:

```
def ma_fonction():
    x = 10  # Variable locale
    print(f"Dans la fonction : x = {x}")

    ma_fonction()
    # print(x)  # ERREUR ! x n'existe pas ici
```

Out[0]:

```
Dans la fonction : x = 10
```

3.4.2 Variables globales

In [0]:

```
## Variable globale
    message = "Bonjour le monde"

    def afficher_message():
    print(message)  # Lecture de la variable globale OK

    afficher_message()
```

Out[0]:

```
Bonjour le monde
```

3.4.3 Le mot-clé `global`

Pour **modifier** une variable globale depuis une fonction, utilisez `global`.

In [0]:

```python
compteur = 0  # Variable globale

    def incrementer():
    global compteur  # Déclare qu'on modifie la globale
    compteur += 1

    print(f"Avant : {compteur}")
    incrementer()
    incrementer()
    incrementer()
    print(f"Après : {compteur}")
```

Out[0]:

```
Avant : 0 Après : 3
```

Évitez les variables globales

L'utilisation excessive de variables globales rend le code difficile à maintenir et à déboguer. Préférez passer des paramètres et retourner des valeurs.

3.4.4 Règle LEGB

Python recherche les variables dans l'ordre **LEGB** :

TABLE 3.1 – *Règle LEGB de résolution des noms*

Lettre	Scope	Description
L	Local	Variables de la fonction courante
E	Enclosing	Variables des fonctions englobantes
G	Global	Variables du module
B	Built-in	Noms prédéfinis Python (`print`, `len`...)

3.5 Fonctions Intégrées (Built-in)

Python fournit de nombreuses fonctions **prêtes à l'emploi**.

TABLE 3.2 – *Fonctions intégrées courantes*

Fonction	Description	Exemple
`len()`	Longueur d'une séquence	`len([1,2,3])` → 3
`type()`	Type d'un objet	`type(42)` → int
`int()`, `float()`, `str()`	Conversion de type	`int("42")` → 42
`abs()`	Valeur absolue	`abs(-5)` → 5
`round()`	Arrondi	`round(3.7)` → 4
`min()`, `max()`	Minimum, maximum	`max(1,5,3)` → 5
`sum()`	Somme d'un itérable	`sum([1,2,3])` → 6
`sorted()`	Tri (nouvelle liste)	`sorted([3,1,2])` → [1,2,3]
`range()`	Séquence de nombres	`range(5)` → 0,1,2,3,4
`enumerate()`	Index + valeur	boucle avec index
`zip()`	Combine des itérables	paires d'éléments
`input()`	Saisie utilisateur	`input("Nom:")`
`print()`	Affichage	`print("Hello")`

In [0]:

```
## Exemples de fonctions intégrées
    nombres = [4, 2, 8, 1, 9, 3]

    print(f"Liste : {nombres}")
    print(f"Longueur : {len(nombres)}")
    print(f"Somme : {sum(nombres)}")
    print(f"Minimum : {min(nombres)}")
    print(f"Maximum : {max(nombres)}")
    print(f"Moyenne : {sum(nombres) / len(nombres):.2f}")
    print(f"Triée : {sorted(nombres)}")
```

Out[0]:

```
Liste : [4, 2, 8, 1, 9, 3] Longueur : 6 Somme : 27 Minimum : 1
Maximum : 9 Moyenne : 4.50 Triée : [1, 2, 3, 4, 8, 9]
```

3.6 Modules Standards

Python dispose d'une **bibliothèque standard** riche. Pour utiliser un module, on l'importe avec `import`.

3.6.1 Le module `math`

In [0]:

```
import math

    # Constantes
    print(f"Pi = {math.pi}")
    print(f"e = {math.e}")

    # Fonctions
    print(f"Racine de 16 = {math.sqrt(16)}")
    print(f"Puissance : 2^10 = {math.pow(2, 10)}")
    print(f"Arrondi sup : ceil(4.2) = {math.ceil(4.2)}")
    print(f"Arrondi inf : floor(4.8) = {math.floor(4.8)}")
    print(f"Factorielle : 5! = {math.factorial(5)}")
```

Out[0]:

```
Pi = 3.141592653589793 e = 2.718281828459045 Racine de 16 = 4.0
Puissance : 2^10 = 1024.0 Arrondi sup : ceil(4.2) = 5 Arrondi inf : floor(4.8) = 4
Factorielle : 5! = 120
```

3.6.2 Le module `random`

In [0]:

```
import random

    # Nombre aléatoire
    print(f"Entier entre 1 et 10 : {random.randint(1, 10)}")
    print(f"Flottant entre 0 et 1 : {random.random():.4f}")
    print(f"Flottant entre 5 et 15 : {random.uniform(5, 15):.2f}")

    # Choix aléatoire dans une liste
    fruits = ["pomme", "banane", "orange", "kiwi"]
    print(f"Fruit aléatoire : {random.choice(fruits)}")

    # Mélanger une liste
    numeros = [1, 2, 3, 4, 5]
    random.shuffle(numeros)
    print(f"Liste mélangée : {numeros}")
```

Out[0]:

```
Entier entre 1 et 10 : 7 Flottant entre 0 et 1 : 0.4521 Flottant
entre 5 et 15 : 11.34 Fruit aléatoire : banane Liste mélangée :
[3, 1, 5, 2, 4]
```

3.6.3 Différentes façons d'importer

In [0]:

```
## Import du module entier
    import math
    print(math.sqrt(25))

    # Import avec alias
    import random as rd
    print(rd.randint(1, 100))

    # Import de fonctions spécifiques
    from math import pi, sqrt
    print(pi)
    print(sqrt(36))

    # Import de tout (à éviter en général)
    # from math import *
```

Out[0]:

```
5.0 42 3.141592653589793 6.0
```

3.7 Fonctions Lambda (Anonymes)

Les fonctions **lambda** sont des fonctions courtes, définies sur une seule ligne, sans nom.

Fonction Lambda

Syntaxe : `lambda paramètres: expression`
Une fonction lambda retourne automatiquement le résultat de l'expression.

In [0]:

```
## Fonction classique
    def carre_classique(x):
    return x ** 2
```

```
    # Équivalent lambda
    carre_lambda = lambda x: x ** 2

    # Utilisation identique
    print(f"Classique : {carre_classique(5)}")
    print(f"Lambda : {carre_lambda(5)}")
```

Out[0]:

```
Classique : 25 Lambda : 25
```

In [0]:

```
## Lambda avec plusieurs paramètres
    addition = lambda a, b: a + b
    print(f"3 + 4 = {addition(3, 4)}")

    # Utilisation courante : tri personnalisé
    etudiants = [("Alice", 85), ("Bob", 92), ("Charlie",
       78)]

    # Trier par note (index 1)
    tries_par_note = sorted(etudiants, key=lambda x: x[1])
    print(f"Par note croissante : {tries_par_note}")

    # Trier par note décroissante
    tries_desc = sorted(etudiants, key=lambda x: x[1],
       reverse=True)
    print(f"Par note décroissante : {tries_desc}")
```

Out[0]:

```
3 + 4 = 7 Par note croissante : [('Charlie', 78), ('Alice', 85),
('Bob', 92)] Par note décroissante : [('Bob', 92), ('Alice',
85), ('Charlie', 78)]
```

3.8 Documentation des Fonctions (Docstrings)

Les **docstrings** documentent vos fonctions pour les autres développeurs (et vous-même !).

In [0]:

```
def calculer_imc(poids, taille):
```

```python
    """
    Calcule l'Indice de Masse Corporelle (IMC).

    Paramètres:
    poids (float): Poids en kilogrammes
    taille (float): Taille en mètres

    Retourne:
    float: L'IMC calculé

    Exemple:
    >>> calculer_imc(70, 1.75)
    22.86
    """
    return poids / (taille ** 2)

    # Accéder à la documentation
    print(calculer_imc.__doc__)

    # Ou avec help()
    # help(calculer_imc)
```

Bonnes pratiques de documentation

- Décrivez ce que fait la fonction (pas comment)
- Documentez les paramètres et leur type
- Documentez la valeur de retour
- Ajoutez des exemples si utile

Récapitulatif : Fonctions en Python

Définition

- `def nom(params):`
- Corps indenté
- Docstring recommandée

Paramètres

- Positionnels : `f(a, b)`
- Par défaut : `f(a=1)`
- Variables : `*args`, `**kwargs`

Retour

- `return valeur`
- Multiples : `return a, b`
- Sans return → `None`

Portée

- Locale → Englobante → Globale → Built-in
- `global` pour modifier une globale

Modules courants

- `math` : calculs mathématiques
- `random` : aléatoire
- `import module as alias`

Lambda

- `lambda x: expression`
- Fonctions courtes anonymes

3.9 Travaux Pratiques du Module 3

Travaux Pratiques : Fonctions

TP 1 – Fonctions de calcul basiques

Créez les fonctions suivantes :

1. `aire_rectangle(longueur, largeur)` : retourne l'aire
2. `perimetre_rectangle(longueur, largeur)` : retourne le périmètre
3. `aire_cercle(rayon)` : retourne l'aire (πr^2)
4. `convertir_celsius_fahrenheit(celsius)` : retourne la température en °F

Testez chaque fonction avec plusieurs valeurs.

TP 2 – Fonctions de validation

Créez des fonctions de validation qui retournent `True` ou `False` :

1. `est_pair(n)` : vérifie si un nombre est pair
2. `est_positif(n)` : vérifie si un nombre est positif
3. `est_majeur(age)` : vérifie si l'âge est ≥ 18
4. `est_email_valide(email)` : vérifie si l'email contient "@" et "."

TP 3 – Fonctions avec listes

Créez des fonctions de traitement de listes :

1. `calculer_moyenne(notes)` : retourne la moyenne d'une liste
2. `trouver_min_max(nombres)` : retourne (minimum, maximum)
3. `compter_occurrences(liste, element)` : compte les apparitions
4. `filtrer_positifs(nombres)` : retourne une liste sans les négatifs

Bonus : N'utilisez pas les fonctions intégrées `min()`, `max()`, `sum()`.

TP 4 – Générateur de mot de passe

Créez une fonction `generer_mot_de_passe(longueur=12, majuscules=True, chiffres=True, speciaux=True)` qui :

1. Génère un mot de passe aléatoire
2. Respecte les options (avec ou sans majuscules, chiffres, caractères spéciaux)
3. Retourne le mot de passe généré

Modules : `random`, `string`

TP 5 – Mini-calculatrice modulaire

Créez une calculatrice avec les fonctions :

1. `additionner(a, b)`, `soustraire(a, b)`, `multiplier(a, b)`, `diviser(a, b)`
2. `calculer(operation, a, b)` : appelle la bonne fonction selon l'opération (+, -, *, /)
3. `menu()` : affiche le menu et gère les saisies utilisateur

Le programme doit tourner en boucle jusqu'à ce que l'utilisateur choisisse de quitter.

3.10 Quiz du Module 3

1 **[Vrai/Faux]** Une fonction sans `return` retourne automatiquement `0`.

☐ Vrai ☐ Faux

2 **[QCM]** Quelle syntaxe définit correctement une fonction ?

A. `function ma_fonction():`

B. `def ma_fonction():`

C. `func ma_fonction():`

D. `define ma_fonction():`

3 **[Vrai/Faux]** `*args` permet de passer un nombre variable d'arguments nommés.

☐ Vrai ☐ Faux

4 **[QCM]** Que retourne `math.sqrt(25)` ?

A. 5

B. 5.0

C. 25

D. Erreur

5 **[QCM]** Comment importer uniquement la fonction `sqrt` du module `math` ?

A. `import sqrt from math`

B. `from math import sqrt`

C. `import math.sqrt`

D. `include math.sqrt`

6 **[Vrai/Faux]** Une variable définie dans une fonction est accessible en dehors de cette fonction.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle est la syntaxe correcte d'une fonction lambda ?

A. `lambda: x * 2`

B. `lambda x: x * 2`

C. `lambda(x): x * 2`

D. `def lambda x: x * 2`

8 **[Question ouverte]** Quelle est la différence entre `print()` et `return` ?

9 **[QCM]** Que signifie LEGB dans le contexte de la portée des variables ?

A. Local, External, Global, Built-in

B. Local, Enclosing, Global, Built-in

C. Loop, Enclosing, Global, Base

D. Local, Enclosing, General, Built-in

10 **[Question ouverte]** Écrivez une fonction `double(x)` qui retourne le double de x.

Table des matières détaillée